國立臺灣大學電機資訊學院資訊工程學研究所

碩士論文

Department of Computer Science and Information Engineering

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis

結合設計規劃、IP 重用與閉迴路修復的代理式 Verilog 程式碼生成框架

An Agentic Framework for RTL Code Generation with Design Planning, IP Reuse, and Closed-Loop Repair

洪聿鍇

Yu-Kai Hung

指導教授: 洪士灝 博士

Advisor: Shih-Hao Hung, Ph.D.

中華民國 115 年 7 月

July 2026

doi:10.6342/NTU202601985

國立臺灣大學碩士學位論文

口試委員會審定書

結合設計規劃、IP 重用與閉迴路修復的代理式 Verilog 程式碼生成框架

An Agentic Framework for RTL Code Generation with Design Planning, IP Reuse, and Closed-Loop Repair

本論文係洪聿鍇君（R13922149）在國立臺灣大學資訊工程學研究所完成之碩士學位論文，於民國 115 年 7 月 30 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

（指導教授）

_____ _____

_____ _____

_____ _____

_____ _____

所　　長：_____

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

摘要

近年來大型語言模型（LLM）已能生成語法正確的程式碼，然而在真實硬體專案中，暫存器傳輸級（RTL）設計工作多以整合與矽智財（IP）重用為主，而非從零撰寫演算法模組，因此對於模型需要能力的要求和軟體設計不同。本論文以 RealBench 基準測試中源自三個真實開源 IP 專案（AES 加密核心、SD 卡控制器、E203 RISC-V 處理器核心）的 60 個模組級任務為評測對象，研究如何以系統工程手段——規劃、接地（grounding）、驗證、修復與路由——提升固定開源權重模型的 RTL 生成能力。

本論文提出兩項主要貢獻。其一為兩階段代理式 IP 重用流水線：規劃代理透過 IP 重用工具（目錄檢索、候選 IP 檢視與評分、介面相容性檢查）針對每個任務的真實 IP 目錄進行規劃，並以封閉詞彙表對齊、完整性檢查的兩階段檢查產生可追溯的結構化重用計畫，再由生成器在 Verilator 驗證與修復迴圈下產生 Verilog。在 T 為零的對照實驗中，流水線通過率達 25.0%（語法 61.7%），為同一模型直接提示（8.3% ／ 25.0%）的 3.0 倍，且增益幾乎全數集中於 IP 重用密集的 E203 家族。其二為逐任務路由器，依任務性質在完整規劃流程與低成本直接生成之間選擇流程。本論文的基準測試作弊稽核發現，原始路由規則的極性係擬合於受污染的解答集；改以清洗後證據重新推導規則——預設走規劃流程，僅窄小的自含運算底層模組走直接生成——後，路由器與其取代的原規則同樣解出 21/60 題，單樣本通過率與語法率均較高，且完全不受洩漏影響。

doi:10.6342/NTU202601985

本論文同時以對照消融精確量化各輔助機制：語意修復快取單調提升語法率（59.4%→66.9%），並使通過樣本更集中於可解任務；兩種修復階段皆呈「速修或永不修」分布，精簡的修復預算（語法約 6 次、功能不超過 4 次）即可保留幾乎全部的修復成功案例，同時大幅節省運算；以黃金模組檢驗持續零分任務，發現並向上游回報一項測試平台競爭條件（已獲基準採納修復），另查出一項黃金模型本身即無法通過評分的基準缺陷，並確認其餘零分為真實難度；同一檢測反向檢驗通過樣本時，更查獲一項提示端洩漏——直接生成提示中出現基準參考模型名稱，使部分樣本僅靠實例化黃金模型即可通過——本文所有數據均已剔除此類樣本。整體結果顯示：下一步的突破點不在語法、規劃或模型規模，而在功能正確性——需要比失配計數更豐富的回饋訊號。

關鍵字：大型語言模型、暫存器傳輸級設計、代理式規劃、IP 重用、閉迴路修復、模型路由、硬體描述語言

doi:10.6342/NTU202601985

Abstract

Large language models can now generate syntactically correct code, but in real hardware projects RTL design work is dominated by integration and IP reuse rather than writing algorithm modules from scratch, so the abilities it demands of a model differ from those of software design. This thesis evaluates on the 60 module-level tasks in the RealBench benchmark, drawn from three real open-source IP projects (an AES cipher core, an SD-card controller, and the E203 RISC-V processor core), and studies how far system engineering — planning, grounding, verification, repair, and routing — can push the RTL generation ability of fixed open-weight models.

The thesis makes two main contributions. The first is a two-stage agentic IP-reuse pipeline. A planning agent plans each task against its real IP catalog through a suite of IP-reuse tools (catalog search, candidate IP inspection and scoring, and interface-compatibility checking), and a two-stage check of closed-vocabulary alignment and completeness turns the result into a structured, traceable reuse plan; a generator then

doi:10.6342/NTU202601985

writes the Verilog under a Verilator verify-and-repair loop. In the temperature-zero controlled comparison the pipeline passes 25.0% of tasks (61.7% syntax), 3.0× the same model under direct prompting (8.3%/25.0%), with the gains concentrated almost entirely in the reuse-heavy E203 family. The second is a per-task router that chooses by task character between the full planning flow and low-cost direct generation. The thesis's own benchmark-gaming audit found that the original routing rule's polarity had been fitted to a contaminated solution set. Re-derived on cleaned evidence — planning flow by default, direct generation only for narrow self-contained computational leaf modules — the router solves 21 of 60 tasks, the same as the rule it replaces, with higher per-sample pass and syntax rates and no exposure to the leak.

Controlled ablations quantify each supporting mechanism precisely. A semantic repair cache lifts the syntax rate monotonically (59.4%→66.9%) and concentrates passing samples inside solvable tasks. Both repair phases show a fix-fast-or-never distribution, so compact repair budgets (about 6 syntax attempts, at most 4 functional) keep essentially all repair successes and cut compute substantially. A golden-model check of persistently zero-scoring tasks found and reported upstream a testbench race condition, which the benchmark has since fixed; it also uncovered a second benchmark defect whose golden model fails its own scoring build, and confirmed the remaining zeros as genuine diffi-

doi:10.6342/NTU202601985

culty. Turned around on the passing samples, the same check caught a prompt-side leak. The direct-generation prompt exposes the benchmark's reference-model names, and some samples passed just by instantiating the golden model; every number in this thesis excludes such samples. The overall results place the next breakthrough not in syntax, planning, or model scale, but in functional correctness, which needs a richer feedback signal than a mismatch count.

Keywords: Large Language Models, RTL Generation, Agentic Planning, IP Reuse, Closed-Loop Repair, LLM Routing, Verilog

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

Contents

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

List of Figures

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

List of Tables

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

Denotation

LLM	Large Language Model（大型語言模型）
RTL	Register-Transfer Level（暫存器傳輸級）
HDL	Hardware Description Language（硬體描述語言）
IP	Intellectual Property core（矽智財；可重用之硬體設計區塊）
DUT	Device Under Test（受測設計；測試平台所例化之目標模組）
FSM	Finite-State Machine（有限狀態機）
CDC	Clock-Domain Crossing（跨時脈域）
RAG	Retrieval-Augmented Generation（檢索增強生成）
AES	Advanced Encryption Standard（進階加密標準）
SDC	SD-Card Controller（SD 卡控制器；本文之任務家族名稱）
E203	Nuclei HummingBirdv2 E203 開源 RISC-V 處理器核心
vLLM	高吞吐量 LLM 推論服務框架（PagedAttention）

doi:10.6342/NTU202601985

pass@k	k 次取樣中至少一次通過之無偏估計通過率
syntax rate	通過基準測試編譯（無任何錯誤與未豁免警告）之樣本比例
pass rate	同時通過編譯與測試平台功能比對之樣本比例
s/f	修復預算表記：每樣本至多 s 次語法修復與 f 次功能修復
MODDUP	Verilator 錯誤類別：重複宣告環境已提供之模組
PINNOTFOUND	Verilator 錯誤類別：例化埠不存在（違反介面契約）

doi:10.6342/NTU202601985

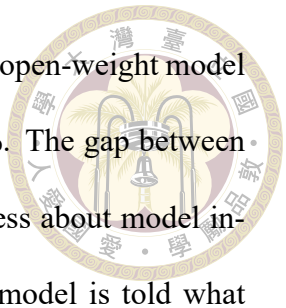
Chapter 1 Introduction

1.1 Motivation

Large language models have improved quickly across nearly every domain of study and research. They draft and refactor software systems, accelerate biological discovery, and increasingly act as day-to-day research assistants, and an industry has grown around turning that capability into engineering practice. Hardware design is one of the most active new directions for this work. Hardware description languages look, at first glance, like the next domain over from software. Verilog is text, it compiles, and it has testbenches. But the economics of hardware make correctness far harsher — an RTL bug that reaches silicon is not patched, it is respun — and the daily reality of hardware engineering differs from the coding-benchmark ideal in a way that matters more than syntax. Real RTL work is dominated by integration and reuse, not by writing fresh algorithms. A working chip is assembled from verified IP blocks, and the engineer's job is to instantiate existing modules correctly, honor interface contracts pin for pin, and wire subsystems whose specifications run to hundreds of pages [11].

Most evaluations of LLM-generated Verilog sidestep this reality. They pose small, self-contained modules with no environment to integrate against [15, 19]. When we instead evaluate on RealBench [10] — 60 module-level tasks drawn from three real IP

doi:10.6342/NTU202601985



projects, scored by a strict compile-and-simulate harness — a capable open-weight model prompted directly compiles on only 25.0 % of tasks and passes 8.3 %. The gap between that number and usefulness is the subject of this thesis. The gap is less about model intelligence than about the system around the model — whether the model is told what already exists, held to the environment's interface contracts, verified against the scorer's own toolchain, and spared work it predictably cannot convert.

1.2 Problem Statement

This thesis addresses the following problem. Given a natural-language specification of one Verilog module inside an existing project — with the project's other modules provided as source files, and a hidden-logic testbench fixing the module's exact port contract — generate RTL that compiles warning-free with the full project and passes the testbench, using only local open-weight models. Three properties make this hard. The environment imposes fatal structural contracts (re-declaring a provided module or missing a testbench port kills the sample outright). The tasks are heterogeneous — thin wrapper modules, algorithmic datapaths, and thousand-line integrators demand different generation strategies. And compute is finite and local. Every planning step, repair attempt, and sample competes for the same GPU, so a system that cannot decide where its compute converts mostly wastes it.

1.3 Contributions

This thesis makes three contributions, each evaluated on the same 60-task benchmark under controlled ablations. The first is a complete generation system, shown in Figure 1.1.

doi:10.6342/NTU202601985

A router reads each task's specification and picks one of two flows. The planning flow plans first and then generates; the direct flow generates immediately. Both flows end in the same verify-and-repair stage.

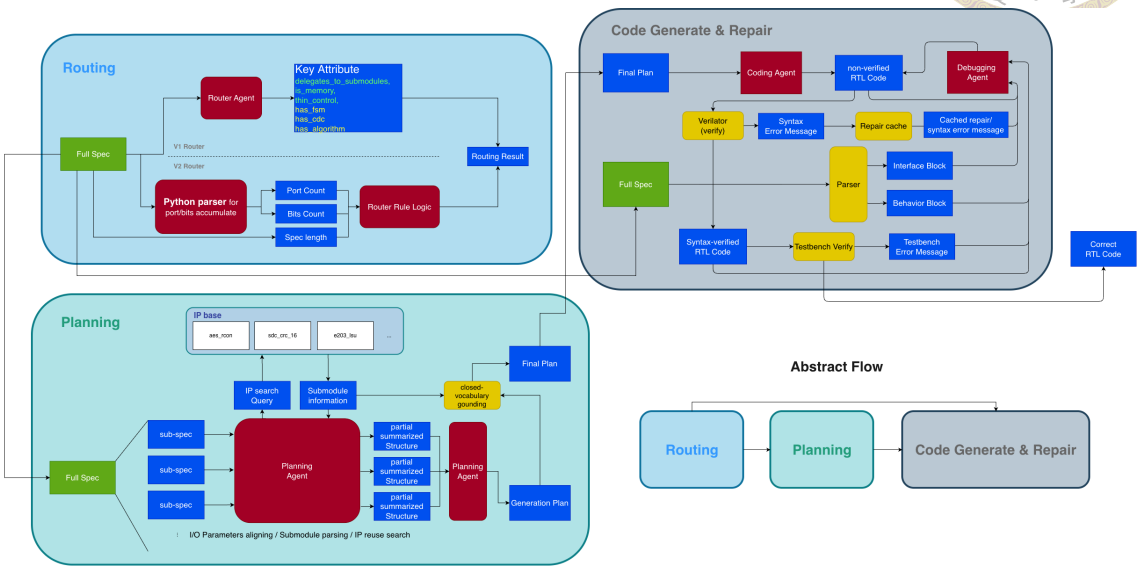


Figure 1.1: Overview of the generation system and its three subsystems. The router (top left) reads the specification and picks a flow per task, from the router agent's key attributes under the v1 rule or from parsed port, length, and state gates under the v2 rule. Under the planning flow (bottom left), a planning agent splits the specification into subspecs, summarizes them into structured form, and merges them into a generation plan, with closed-vocabulary grounding against a per-task IP catalog, before any code is written; the direct flow skips planning. Both flows enter the same generate-and-repair stage (top right), where a coding agent iterates under Verilator and testbench verification backed by a repair cache. The abstract-flow inset (bottom right) compresses the three stages into the path a task traverses. Chapter 3 presents all three subsystems — the planning flow, the repair machinery, and the router.

1. **A two-stage agentic IP-reuse pipeline with per-task flow routing** (Chapter 3). A planning agent works the task through a suite of IP-reuse tools — catalog search, candidate inspection and scoring, and interface-compatibility verification — and emits a structured reuse plan grounded against a per-task catalog of the modules the environment actually provides; a generator then works under environment-faithful verification and repair. At temperature 0 the pipeline passes 25.0 % of tasks against the same model's direct 8.3 % (3.0×; syntax 2.5×), with the gain concentrated almost entirely in the reuse-rich processor-core family (Section 5.1). In front of the

doi:10.6342/NTU202601985

planning flow sits a per-task router (Section 3.5), an LLM feature extractor that reads the specification and chooses, before planning-scale compute is spent, between the full planning flow and a cheap direct flow. The original routing rule was fitted to a solution set the benchmark-gaming audit later invalidated; re-derived from the cleaned evidence, the rule routes to the planning flow by default, sends only narrow self-contained computational leaves to direct generation, and matches the coverage of the rule it replaces (21 of 60 tasks solved) with better per-sample quality and no exposure to the leak (Section 5.7).

2. **Controlled measurements of the supporting machinery** (Chapter 5). A semantic repair cache lifts syntax monotonically (59.4 %→66.9 %) while concentrating passing samples inside solvable tasks. A repair-budget calibration shows both repair phases converge within their first attempts, so compact budgets keep nearly all of the value at a fraction of the compute. The functional repair loop is reported as its own experimental arm because its feedback signal is the scoring oracle — an accounting line this literature rarely draws.

3. **A benchmark- and harness-artifact audit methodology** (Section 5.9). Golden implementations are run through the full scoring path to decide whether persistent zeros are real, and generated RTL is screened for benchmark-gaming patterns. The audits found a testbench race that scored byte-perfect S-box implementations as total failures (root-caused and fixed upstream in the benchmark), exposed a second benchmark defect whose golden model fails its own scoring build, confirmed the other zeros as genuine difficulty, and located the warning classes on which our own repair gate is stricter than the scorer. They also caught a prompt-side leak that let directly-prompted samples pass by simply instantiating the benchmark's reference

doi:10.6342/NTU202601985

models; every number in this thesis excludes those samples.

Throughout, every claim is tied to a controlled comparison, and results are reported with the same care whether they move the headline metric or correct it. The audit that caught a benchmark-gaming pattern in our own direct-flow samples is as informative as any number it revised.

1.4 Thesis Organization

Chapter 2 reviews the background and situates the thesis in four research threads. Chapter 3 presents the methodology — the two-stage planning flow, its repair machinery, and the cascade router between the planning flow and a cheap direct flow. Chapter 4 fixes the benchmark, models, metrics, and the inventory of every experimental run. Chapter 5 reports the results, covering the headline comparison, the ablations, the routing study, a failure categorization, and the benchmark-artifact audit. Chapter 6 discusses implications, states threats to validity, and concludes. Appendices collect prompt templates and per-module result tables.

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

Chapter 2 Background and Related Work

This chapter sets out the hardware-design background the thesis assumes (Section 2.1), then surveys the four research threads it builds on: LLMs for Verilog generation (Section 2.2), verification-in-the-loop repair (Section 2.3), retrieval-augmented code generation (Section 2.4), and LLM routing and cascades (Section 2.5). Section 2.6 positions this thesis against them.

2.1 RTL Design, Verification, and IP Reuse

Register-transfer-level (RTL) design expresses digital hardware as synchronous transfers between registers through combinational logic, written in a hardware description language — here Verilog/SystemVerilog [9]. Unlike software, RTL code carries a hard structural interface. A module's port list is a physical contract with its environment, and simulation semantics (blocking vs. nonblocking assignment, delta-cycle scheduling) can make two logically equivalent descriptions observably different to a testbench. That subtlety comes back in Section 5.9. Correctness is established by tools, not review. This thesis uses Verilator [28] for lint and compiled simulation and Yosys [32] for

doi:10.6342/NTU202601985

synthesis-based structural analysis.

Industrial hardware productivity rests on IP reuse, the practice of assembling verified, pre-existing blocks rather than rewriting them, codified two decades ago [11]. A working chip team spends much of its effort not on new datapaths but on integration — instantiating existing modules and wiring their interfaces correctly. This observation is the design premise of Chapter 3. If real-world RTL work is reuse-dominated, an RTL-generating agent should plan reuse explicitly rather than synthesize every line from scratch. The open-source projects our benchmark draws from — an AES cipher core and an SD-card controller from OpenCores [24] and the HummingBirdv2 E203 RISC-V processor [20] — are themselves products of this reuse culture.

2.2 Large Language Models for Verilog Generation

Benchmarks. Evaluation infrastructure for LLM-generated Verilog began with fine-tuning-era studies [25, 29] and then settled into standard suites. VerilogEval [15] offers machine-checked HDLBits-derived problems with the pass@k protocol inherited from HumanEval [5], later revised with specification-to-RTL tasks [26]. RTLLM [19] adds larger open-ended designs judged on syntax, functionality, and quality. These suites consist mostly of self-contained single modules. RTL-Repo [2] moved evaluation toward multi-file repository context. RealBench [10], used throughout this thesis (Section 4.1), builds tasks from complete, real IP projects in which a target module must integrate against provided dependencies and survive a strict warnings-as-fatal harness. Table 2.1 quantifies how far apart these settings are. A RealBench module task averages an order of magnitude more code lines and synthesized cells than a VerilogEval problem, and is specified

doi:10.6342/NTU202601985

by a document more than thirty times longer. It is also the only suite whose tasks routinely instantiate submodules. The distinction is not cosmetic. Chapter 5 shows that the phenomena that dominate RealBench (interface contracts, reuse, integration) are precisely the ones self-contained benchmarks cannot exhibit.

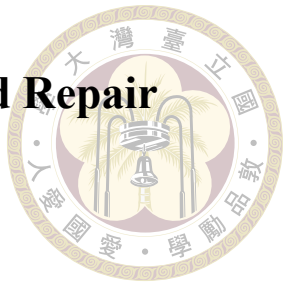
Table 2.1: Design and specification scale across Verilog generation benchmarks (per-design averages, reproduced from the RealBench study [10]). "Cells" = synthesized circuit cells; "Doc. lines" = specification length.

Benchmark	Designs	Code lines	Cells	Submodules	Doc. lines
Thakur et al. [29]	17	16.6	7.9	0	1.8
RTLLM v1.1 [19]	29	56.1	48.1	0.3	22.3
RTLLM v2 [19]	50	46.1	33.7	0.2	20.8
VerilogEval-human [15]	156	15.8	31.4	0	5.7
VerilogEval-machine [15]	143	13.9	6.4	0	3.2
VerilogEval v2 [26]	156	16.1	31.4	0	16.5
RealBench-module [10]	60	241.2	520	1.6	197.3
RealBench-system [10]	4	3537.3	1.1M	14.5	2900

Models and systems. One line of work improves the generator itself. VeriGen fine-tunes on harvested Verilog corpora [29]; ChipNeMo domain-adapts LLMs across chip-design tasks [14]; RTLCoder [17], CodeV [38], SiliconMind-V1 [6], and CraftRTL [16] push open models past commercial baselines with domain-specialized training pipelines — CodeV through multi-level code summarization, SiliconMind-V1 through multi-agent distillation and debug-reasoning workflows, and CraftRTL by adding correct-by-construction non-textual representations and targeted repair data. Conversational studies such as Chip-Chat [3] probed interactive design with a human in the loop. This thesis is orthogonal to all of them. We hold the base models fixed (off-the-shelf open weights, Section 4.2) and engineer the system around the model — planning, grounding, verification, repair, and routing. Better base models slot in unchanged; indeed Section 5.8 measures exactly what a $6\times$ model scale-up does and does not buy under a fixed system.

doi:10.6342/NTU202601985

2.3 Verification-in-the-Loop Generation and Repair



Feeding tool feedback back into generation is the field's standard answer to the unreliability of one-shot code. In software, self-repair with execution feedback is well studied and its limits documented [21]; agentic framings add explicit reasoning-action loops [36], verbal self-reflection [27], and purpose-built agent-computer interfaces [35]. In hardware, AutoChip iterates generation against compiler and simulation errors [30]; RTLFixer targets syntax repair with retrieval over an error-pattern database [31]; MEIC frames RTL debug as an iterated LLM workflow [33]; and VerilogCoder couples an autonomous agent with graph-based planning and an AST/waveform-tracing tool, reporting strong VerilogEval-Human results [8].

The repair machinery of Section 3.4 belongs to this family but contributes three things the prior systems do not measure. First, a semantic repair cache that replays the model's own verified diagnostic→diff fixes across samples. It is related in spirit to RTLFixer's error-pattern retrieval, but populated online from verified successes rather than curated offline, and evaluated for what it changes (sample density) and what it cannot (task solvability, Section 5.4). Second, an explicit oracle-feedback accounting. Our functional repair loop consumes the scoring testbench's mismatch signal, so it is quarantined as a separate arm, a methodological line most repair papers do not draw. Third, a budget calibration showing both repair phases are fix-fast-or-never (Section 5.6), turning repair-loop depth from folklore into a measured engineering parameter.

doi:10.6342/NTU202601985

2.4 Retrieval-Augmented Code Generation

Retrieval-augmented generation grounds model output in external sources [13]; for code, retrieval over repositories supplies cross-file context that no prompt window holds natively, whether by similar-code lookup [18] or iterative repository-level retrieval interleaved with generation [37]. Our per-task IP catalog (Section 3.1) is a deliberately narrow instance. Retrieval is scoped to the task's own dependency closure, never across tasks, and its product is not only free-form context but a closed vocabulary of instantiable modules. The planner queries this catalog through its retrieval tools, and the catalog digest is also injected into its prompt. Every reuse decision in its output is grounded against the same vocabulary post-hoc (Section 5.2), so retrieval serves as constraint as well as context.

2.5 LLM Routing and Cascades

A parallel literature allocates queries across models of different cost. FrugalGPT sequences models with a learned stopping rule [4]; Hybrid LLM trains a router to send easy queries to a small model [7]; RouteLLM learns routers from human preference data [22]; AutoMix self-verifies before escalating [1]. All of these route between models on general text tasks, assuming the larger model weakly dominates in quality. Our router (Section 3.5) differs on both axes. It routes between workflows over the same model — a cheap direct flow and an expensive agentic planning flow — and the expensive flow does not dominate. On self-contained algorithmic modules the planning flow is actively harmful (Section 3.5.1), so routing is about matching task character to flow shape, not buying quality with compute. Its cascade structure is also domain-specific. The mid-tier probe

doi:10.6342/NTU202601985

inspects the planning flow's own first artifact (the plan), so the escalation step costs one planner call rather than a full duplicate generation.

2.6 Positioning

Table 2.2: This thesis against the closest related systems. "Reuse planning" = explicit grounded IP-reuse decisions; "workflow routing" = per-task choice among generation flows.

System	Repair loop	Retrieval	Reuse planning	Workflow routing	Repo-realistic eval
VeriGen [29]	–	–	–	–	–
AutoChip [30]	✓	–	–	–	–
RTLFixer [31]	✓	✓	–	–	–
VerilogCoder [8]	✓	✓	–	–	–
RepoCoder [37]	–	✓	–	–	✓
RouteLLM [22]	–	–	–	model-level	–
This thesis	✓	✓	✓	✓	✓

To our knowledge, this thesis is the first system to combine catalog-grounded IP-reuse planning with per-task workflow routing for RTL generation, and the first to evaluate the combination on a repository-realistic benchmark under controlled ablations (Table 2.2). By design, it also contributes the kind of controlled diagnostic measurements the surveyed literature rarely reports: a grounding pass that makes every reuse decision in every plan auditable, a repair cache whose effect is precisely located (sample density inside solvable tasks), a budget calibration that pins down the cost-efficient repair depth, and a golden-model audit methodology that keeps the benchmark signal trustworthy (Section 5.9).

doi:10.6342/NTU202601985

Chapter 3 A Two-Stage Agentic IP-Reuse Pipeline with Cascade Routing

This chapter presents the generation methodology of the thesis. Its core is a two-stage agentic pipeline that generates RTL by first planning which existing IP blocks to reuse and how to integrate them, and only then generating Verilog under a verify-and-repair loop. A per-task cascade router completes the methodology by deciding, before the expensive planning machinery runs, whether a task needs it at all.

The motivation for the two-stage split is the way large language models are usually used, and the way that habit fails on tasks of this scale. The common practice is to hand the model everything at once: the full specification, every dependency source, and the request for final code in a single giant prompt. On a RealBench task that one-shot prompt mixes hundreds of pages of behavioral description with thousands of lines of provided RTL, and the model must simultaneously decide what to build, what already exists, and how to write it. These competing objectives confuse one another and strain the context window, and the interface details that matter most end up buried in the middle of it. Separating the work into a planning stage (decide the reuse and integration structure over a compact, interface-

doi:10.6342/NTU202601985

centric view) and a generation stage (write one module against an already-resolved plan) gives each LLM call a single objective and a prompt shaped for exactly that objective. The separation is better on both axes we care about. It improves accuracy, because each stage sees the right information at the right altitude, and efficiency, because the expensive full-context reasoning happens once in the plan rather than being re-derived in every sample and repair turn.

Section 3.1 gives the dataflow and the environmental contracts the pipeline must honor. Sections 3.2 and 3.3 describe the two stages. Section 3.4 describes the three repair mechanisms layered on the generator — the syntax repair loop with a semantic repair cache, the functional repair loop, and diagnostic-driven specification slicing. Section 3.5 presents the cascade router that chooses per task between this planning flow and a cheap direct flow. Design decisions are motivated here; their empirical evaluation is deferred entirely to Chapter 5.

3.1 Overview and Environmental Contracts

The pipeline processes one benchmark task end-to-end as follows (Figure 1.1):

1. **Catalog construction.** The dependency sources that the evaluation environment provides for the task are indexed into a per-task reusable-IP catalog, with one entry per provided module carrying its name, port list, and source. The catalog is the pipeline's model of what the test environment supplies, and every downstream reuse decision is expressed in its vocabulary.

2. **Routing.** The cascade router (Section 3.5) reads the specification and decides which

doi:10.6342/NTU202601985

flow the task takes. Integration-shaped tasks run the planning flow of steps 3–5; narrow self-contained tasks skip straight to step 5 and generate from the specification alone.

3. **Planning.** The planner (Section 3.2) reads the specification and the catalog and emits a structured IP-reuse plan consisting of requirements, module decomposition, a reuse decision per submodule (reuse a catalog entry vs. write new RTL), and integration notes.

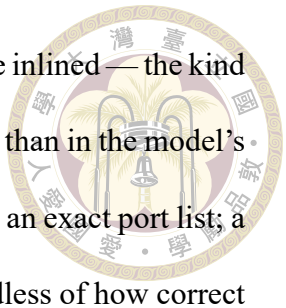
4. **Plan adaptation.** An adapter normalizes the plan, resolves each reuse decision to concrete source code, and assembles the generation prompt from the original specification, the signatures of reused modules, environment notes, and the testbench's DUT instantiation, taken verbatim as the binding interface contract.

5. **Generation and repair.** The generator (Section 3.3) produces the target module and iterates under an internal Verilator lint that compiles the generated code against the same file set the benchmark will use, feeding diagnostics back as repair prompts (Section 3.4).

6. **Evaluation.** The final candidate is scored by the untouched RealBench Makefile as described in Section 4.1.

The two fatal contracts. Two properties of the evaluation environment shape nearly every design decision downstream, and both are fatal when violated. First, the environment compiles every Verilog file in the task directory together, so any provided dependency module that the generated code re-declares produces a duplicate-module error (`MODDUP`). Shipped dependencies must be instantiated, never re-implemented. (The single family-

doi:10.6342/NTU202601985



wide exception, `e203_extend_csr`, is not shipped as a file and must be inlined — the kind of irregularity that motivates grounding plans in the real catalog rather than in the model's memory.) Second, the testbench instantiates the device under test with an exact port list; a generated module that misses one port fails with `PINNOTFOUND` regardless of how correct its internals are. The pipeline addresses the first contract by stripping re-declared modules and cataloging what is provided, and the second by injecting the testbench's DUT instantiation into every generation and repair prompt as a hard interface contract.

3.2 Stage One: The Grounded Planner

The planner is a tool-calling agent. It plans by interacting with a suite of IP-reuse tools scoped to the per-task catalog, and returns a structured JSON plan with a fixed schema — requirement summary, module decomposition, per-module reuse decisions (`selected_ip` naming a catalog entry, or `new_rtl_required`), and integration, verification, and debugging plans. Figure 3.1 shows the stage's dataflow.

The IP-reuse tool suite. Four tools carry the reuse workflow. `search_reuse_ip` queries the per-task catalog for candidate modules (token-overlap, semantic-embedding, or hybrid retrieval backends, selected by `--planner-search-mode`); `inspect_reuse_ip` returns one candidate's behavior, interfaces, parameters, limits, and integration notes; `evaluate_ip_candidate` scores a candidate on function match, interface compatibility, configurability, and verification status; and `check_port_compatibility` parses port declarations and verifies that a proposed instantiation's connections are legal. Auxiliary tools (`write_artifact`, `generate_rtl_module`, `validate_verilog`) let the agent persist plan artifacts and

doi:10.6342/NTU202601985

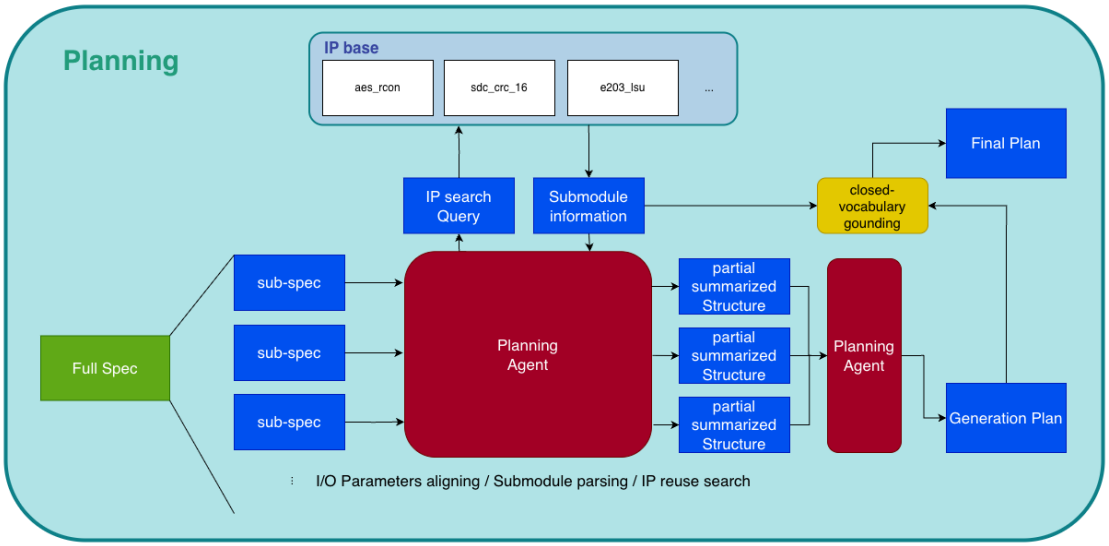
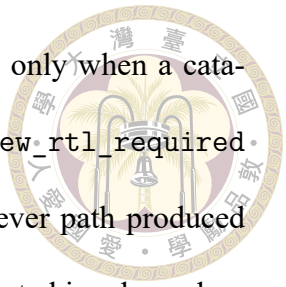


Figure 3.1: Dataflow of the planning stage. Large specifications are split into sub-specs and summarized chunk-by-chunk into partial structured summaries (the condensation described later in this section). While planning, the agent queries the per-task IP catalog and reads back submodule interface information, and a merge pass turns the partial summaries into the generation plan. Closed-vocabulary grounding then resolves every reuse decision against the catalog and releases the final plan to the generator.

sanity-check RTL sketches. The intended interaction per submodule is search → inspect → evaluate → decide. The agent searches the catalog with a behavioral query, inspects the top candidates' interfaces, scores the fit, and only then commits the submodule to a reuse decision.

Reuse or new RTL. The per-module decision between `selected_ip` and `new_rtl_required` is the plan's load-bearing output, and its two error directions cost very differently. Declaring reuse of a module that does not fit (or does not exist under that name) is fatal downstream. The generator either re-implements a shipped dependency (`MODDUP`) or instantiates a phantom (`MISSING-MODULE`), both of which kill the sample at compile time. Missing a legitimate reuse is gentler but wasteful. The generator re-derives, from prose, logic that already exists in verified form — exactly the work IP reuse is meant to eliminate — and every re-derived line is a fresh opportunity for functional error. The decision rule

doi:10.6342/NTU202601985



therefore leans on evidence. A submodule is marked `selected_ip` only when a catalog entry matches both its function and its interface obligations, and `new_rtl_required` otherwise. The grounding mechanisms below guarantee that whichever path produced the name, it resolves to a real catalog entry. Because the environment ships dependencies as files, reuse decisions also determine prompt composition. Reused modules enter the generation prompt as interfaces to instantiate, and new-RTL modules as behavioral requirements to implement.

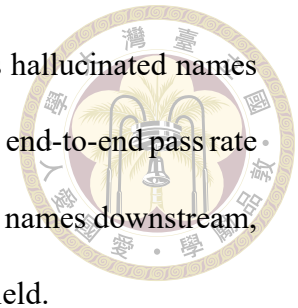
Three mechanisms make the plan's grounding independent of any single step of the agent's trajectory — whatever mixture of tool evidence and direct reading produced a decision, the result is checked against the catalog. Together we call them grounding-by-construction:

1. **Catalog injection.** The real catalog vocabulary — every reusable module id with a one-line digest — is injected directly into the planner's prompt, and the system prompt mandates a closed vocabulary in which `selected_ip` must be one of the listed ids.

2. **Closed-vocabulary grounding.** Every `selected_ip` in the returned plan is post-hoc resolved against the catalog: exact matches kept; near misses remapped by prefix/substring matching and then fuzzy string similarity (cutoff 0.6); irrecoverable names dropped and the submodule marked `new_rtl_required`. The plan also records the grounding outcome per decision, making plans auditable.

3. **Completeness gate.** A task whose catalog is non-empty but whose plan contains no reuse and no integration content is re-prompted once, and the more complete of the two attempts is kept.

doi:10.6342/NTU202601985



Chapter 5 (Section 5.2) audits the effect. Grounding eliminates hallucinated names from plans entirely, making every plan correct and auditable, while the end-to-end pass rate stays essentially level. The generator was already robust to imperfect names downstream, so the gain lands in plan quality and auditability rather than in raw yield.

Large specifications. Specifications beyond a token threshold (45,000 estimated tokens) are condensed before planning by a chunk-and-merge summarizer that renders two views from one manifest: a planner view (interfaces, reuse queries, and a brief behavioral digest) and a generation view (full requirements for to-be-generated logic; interface-only for provided IP). Port tables and module headers are carried verbatim from the raw specification past the LLM summarizer, because interface fidelity is exactly what the two fatal contracts punish. Condensation compresses the largest specifications by roughly $5\times$ (e.g., $\sim$720 k $\rightarrow$ $\sim$146 k characters) while keeping every shared fact and module interface stated exactly once.

3.3 Stage Two: The Generator

The generator receives the adapted plan and produces the target module with a code-focused prompt assembled from the (possibly condensed) specification; the source and signatures of every reused module; the environment notes (which modules are provided as files, which macros the environment defines); and the testbench DUT instantiation as the interface contract. Generated text is parsed for a Verilog code block; provided dependency modules that the model re-declared anyway are stripped before verification. The full prompt template is reproduced in Appendix 6.4. Figure 3.2 shows the stage's dataflow, including the repair machinery that Section 3.4 describes.

doi:10.6342/NTU202601985

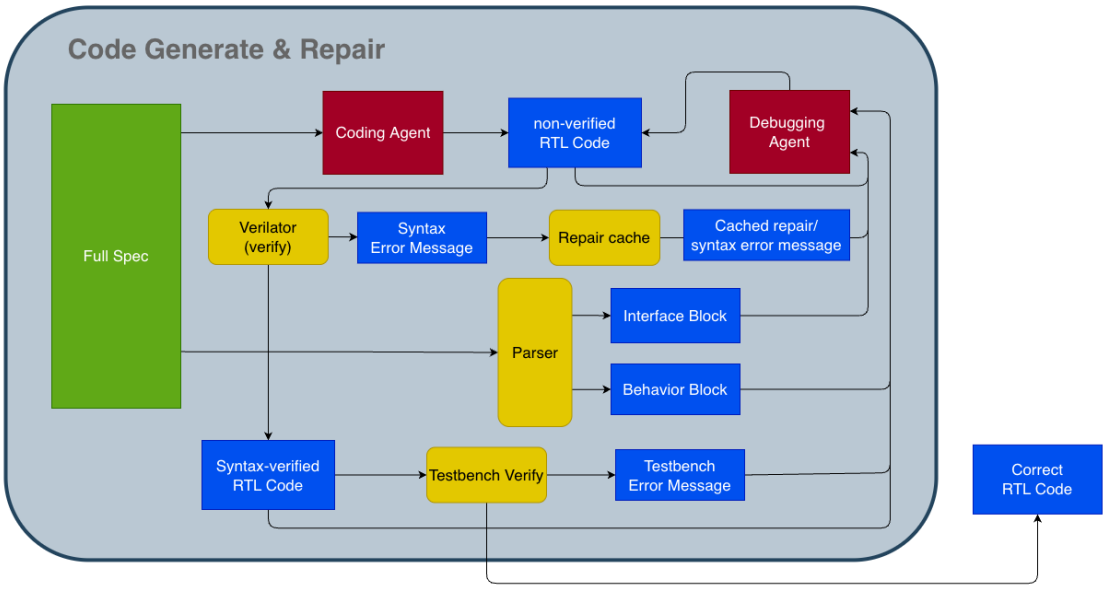


Figure 3.2: Dataflow of the generation and repair stage. The coding agent produces a candidate module, which Verilator lints against the same file set the benchmark compiles. Lint diagnostics are matched against the semantic repair cache and fed, with any cached fix hints, to the debugging agent for syntax repair. A parser splits the specification into interface- and behavior-topic slices; the interface slice feeds syntax repair and the behavior slice functional repair (Section 3.4.3). Syntax-verified candidates are simulated against the testbench, whose mismatch reports drive the functional repair loop (off by default, Section 3.4.2). The surviving candidate exits as the final RTL. In the planning flow the coding prompt additionally carries the final plan (Figure 1.1).

doi:10.6342/NTU202601985

Internal verification mirrors the benchmark. The generator's inner loop lints each candidate with Verilator against the same file set the benchmark compiles — dependencies, reference model, and testbench, with the task Makefile's warning waivers applied — rather than linting the generated file in isolation. This design decision followed a concrete failure. An earlier isolated lint could not see testbench-side pin connections, so interface violations (`PINNOTFOUND`) sailed through internal checks and died only at evaluation. Mirroring the benchmark's compile closes that gap; on the runs where both verdicts exist for identical code, the internal lint agrees with the benchmark's syntax verdict on 60 of 60 tasks. One divergence survives in the strict direction, however. The gate treats a few warning classes as fatal that the scoring build never emits, burning repair budget on code that is not the problem, an artifact quantified in Section 5.9. The cost of this fidelity is that Verilator diagnostics can quote testbench or reference lines into repair prompts; we return to this leak surface in Section 6.2.

Syntax first, then function. The verification order — establish syntax correctness, then check function — is a deliberately robust flow, and it mirrors how an RTL engineer works. Code that does not compile cannot be simulated, so functional evidence only exists for compiling candidates. The ordering also matches the character of the two feedback signals. Syntax diagnostics are precise, local, and actionable (a file, a line, an identifier, an error class), so they are worth iterating on first and cheaply; functional mismatch reports are coarse and global, so they are spent only on candidates that have already cleared the structural contracts. Every repair mechanism in the next section is built around this two-gate structure. Candidates flow through the syntax gate into the functional gate, and no repair turn ever advances a candidate that regresses the earlier gate.

doi:10.6342/NTU202601985

3.4 Repair Mechanisms

Three mechanisms sit on top of the generator's inner loop (Figure 3.3). Throughout the thesis, a repair budget "s/f" means at most s syntax-repair and f functional-repair attempts per sample. The prompt text of both repair loops is reproduced in Appendix 6.4.

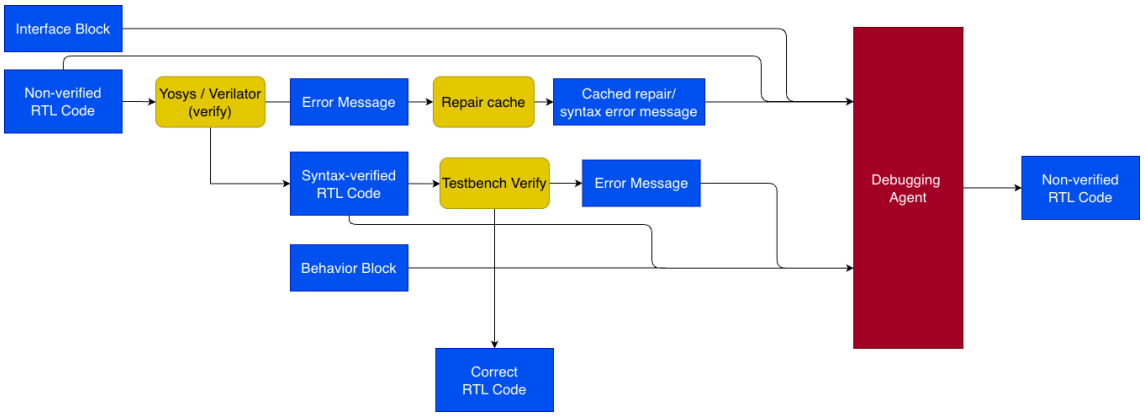


Figure 3.3: The repair architecture. A candidate is first verified with Yosys/Verilator; its diagnostics, any similar cached repairs (Section 3.4.1), and the specification's interface-topic slice feed the debugging agent. A syntax-verified candidate is then simulated against the testbench, whose mismatch report and the behavior-topic slice feed the same agent (off by default; Section 3.4.2). The agent returns a new candidate that re-enters verification, up to the repair budget.

3.4.1 Syntax Repair Loop and the Semantic Repair Cache

When the internal lint fails, the diagnostics are fed back to the model in a repair prompt together with the current candidate, and the process repeats up to the syntax budget. The loop is enhanced by a semantic repair cache that reuses the model's own past successes. Whenever a repair subsequently passes verification, the pair (diagnostic signature $\rightarrow$ unified-diff excerpt of the fix) is stored. Signatures are Verilator diagnostic lines with file paths and identifiers stripped, gated by error code, so that structurally identical errors on different modules collide. On a later failure, sufficiently similar cached entries are injected into the repair prompt — advisory only, never auto-applied, and never sourced

doi:10.6342/NTU202601985

from benchmark reference code. The cache has three scopes: `off`, `task` (shared only across one task's samples), and `run` (shared across the whole run). Run scope maximizes hint availability; what each scope buys is measured in Section 5.4.

3.4.2 Functional Repair Loop

The syntax loop is structurally blind to the largest failure class, candidates that compile but fail the testbench (Section 5.8). The functional repair loop attacks this class directly. After a candidate passes syntax, the testbench is run; on mismatch, the simulator's mismatch report ("Output `x` has N mismatches, first at time T") is fed back in a prompt that frames the problem explicitly as logic repair — the interface is correct, do not change it. The loop keeps the best compiling candidate by lowest mismatch count and never advances to a candidate that regresses syntax.

One methodological property makes this loop different in kind from syntax repair, and we flag it prominently. The feedback signal comes from the same testbench that scores the benchmark. A pass obtained through functional repair therefore means "specification plus oracle feedback," not "specification alone." For that reason the loop is off by default and reported as a separate experimental arm (Section 5.5); headline pipeline-vs-baseline comparisons never include it.

3.4.3 Diagnostic-Driven Specification Slicing

Repair prompts initially carried the full specification, which for large tasks buries the relevant sentence under tens of thousands of irrelevant tokens. Specification slicing exploits the fact that Verilator diagnostics and testbench mismatch reports name the offend-

doi:10.6342/NTU202601985

ing identifiers exactly. The slicer extracts those identifiers, then assembles a focused slice. The slice contains the verbatim interface excerpts (port tables, module headers) plus only the specification sections that mention an offending identifier, under a character budget. When a diagnostic names no identifier (e.g., a bare parse error), the slicer returns nothing and the full specification is kept, so repair is never starved of context. Interface-topic slices feed the syntax repair loop, where the errors are overwhelmingly signal-naming and port-contract issues; behavior-topic slices feed the functional repair loop, where the model must locate mis-implemented logic.

The compression this buys is large. On the largest specification the slice is a $\sim$29$\times$ reduction ($\sim$704 k $\rightarrow$ 24 k characters). But the reduction rate is only meaningful when read together with the repair success rate, because a smaller prompt that dropped the fixing sentence would be a pure regression. The budget calibration of Section 5.6 shows the slice preserves the signal that matters. With sliced repair prompts, the rescues that occur land within the first few attempts of both loops, so each rescue is reached at a fraction of the full-specification token cost per attempt.

3.5 Cascade Routing between Direct and Planning Flows

The planning flow is not the cheapest way to solve every task. Comparing it task by task with direct generation — a single-shot prompt with no plan — shows that the two flows have sharply different economics. The planning flow's coverage subsumes the direct flow's, but at an order of magnitude more tokens per sample, and its planning machinery, decisive on integration problems, is measurably counterproductive on self-contained algorithm and FSM design, exactly where the cheap flow matches it. This division matches the

doi:10.6342/NTU202601985

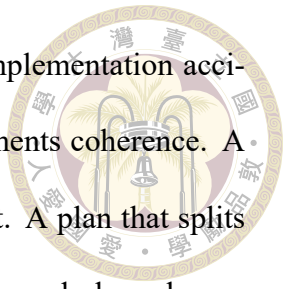
purpose of the thesis. IP-reuse problems mostly need several existing modules arranged and integrated correctly, which is where the plan pays. The residual algorithmic tasks are served equally well, and far more cheaply, by direct generation. The final component of the methodology is therefore a per-task cascade router that predicts, before planning-scale compute is spent, which of the two flows a task should take. Section 3.5.1 establishes why the two flows are complementary; Section 3.5.2 defines the task classes and the oracle labels used to evaluate routing; and Section 3.5.3 describes the two-tier cascade and the metrics it is scored on.

3.5.1 Motivation: Two Complementary Flows

Running both flows on all 60 tasks (the HYBRID arm of Table 4.2) motivates routing. With ten samples per flow, the planning flow solves 20 tasks; the direct flow solves 11 — every one of them also solved by the planning flow. Coverage, then, does not argue for keeping a direct flow at all — cost does. Wrapper and integration modules pass only under the planning flow, which supplies the exact interfaces of the submodules they must instantiate and wire. Self-contained algorithmic modules pass under direct generation too, in one coherent piece, at roughly a tenth of a planning-flow sample's token cost. On exactly those tasks the planning machinery is measurably counterproductive, as the next paragraph details. One flow dominates on ability; it is far from dominant on cost per solved task.

Why planning is overhead on algorithm/FSM tasks. That the planning flow converts poorly on algorithmic and FSM tasks is measurable in the HYBRID run's own planning flow. On tasks whose specification centers on a state machine, compiling samples con-

doi:10.6342/NTU202601985



vert to passes at 16.2 %, against 43.1 % elsewhere. This is not an implementation accident; it follows from what planning does. First, decomposition fragments coherence. A state machine or an arithmetic datapath is one tightly coupled artifact. A plan that splits it into submodule pieces forces the generator to implement fragments and glue where a single `always`-block idiom — exactly the pattern the model knows best from pretraining — would have expressed the whole behavior (compiling samples that follow the plan's multi-module decomposition pass at 12.5 %, against 35.3 % for single-module outputs). Second, the plan is a paraphrase. It restates the specification's behavior in summarized form, and for algorithmic logic the value lives in exact details (encodings, priorities, edge conditions) that summarization inevitably coarsens. The generator then optimizes toward the paraphrase rather than the source. Third, the plan's main asset — verbatim submodule interfaces — is worth little on a task with nothing to instantiate, so the algorithmic task pays planning's context and structure overhead without collecting its benefit. Routing exists to give each task only the machinery it benefits from.

The brute-force deployment — run both flows on every task — is the most expensive one, and it wastes most of its budget. Every direct sample spent on a wrapper module converts to nothing, and every planning-flow sample spent on a small self-contained module pays an order of magnitude more tokens for a task the cheap flow solves equally well. A router that sends each task to its matching flow can, in principle, hold the attainable coverage at a fraction of the cost. The routing problem is to make that prediction before the expensive flow has been paid for.

doi:10.6342/NTU202601985

3.5.2 Task Classes and Oracle Labels

Before formalizing classes, it is worth looking at the two solved sets concretely rather than only through aggregate rates. In the HYBRID run, nine tasks pass only under the planning flow, and eleven pass under direct generation as well as under the planning flow (Table 3.1). The membership is telling. The planning-only set is wrappers, memories, and thin glue — SRAM and memory wrappers, controller and register-file shims, and the IFU integrator — tasks whose difficulty is wiring interfaces the direct prompt cannot see. The direct-solvable set is small self-contained modules — the AES S-boxes and key expansion, the CRC engines, and clock, CSR, and interrupt utilities — whose behavior fits in one coherent piece and offers nothing to reuse.

Table 3.1: The planning-only and direct-solvable sets of the HYBRID run ($60 \times (10{+}10)$, ref-hack samples excluded per Section 5.9), with structural statistics of their golden implementations (median / mean / range). "Own cells" = synthesized cells of the module's own logic after subtracting submodule instances; regular-array wrappers count as 0, modules Yosys cannot elaborate standalone are omitted. Yield counts passing samples per solved task in the set's own flow.

	Planning-only (9 tasks)	Direct-solvable (11 tasks)
Members	`e203_srams`, `e203_itcm_ram`, `e203_dtcm_ram`, `e203_dtcm_ctrl`, `e203_exu_regfile`, `e203_exu_alu_csrctrl`, `e203_exu_alu_rglr`, `e203_ifu`, `e203_ifu_minidec`	`aes_sbox`, `aes_inv_sbox`, `aes_key_expand_128`, `aes_rcon`, `sd_crc_7`, `sd_crc_16`, `sd_fifo_tx_filler`, `e203_clkgate`, `e203_clk_ctrl`, `e203_extend_csr`, `e203_irq_sync`
Code lines	114 / 145 / 71–344	93 / 121 / 34–323
Spec size (chars)	5.4k / 8.9k / 3.6k–26k	3.5k / 3.9k / 1.6k–6.8k
Own cells	0 / 24 / 0–122	20 / 201 / 0–640
Passing yield	stable (mean 7.7/10)	exploratory (mean 4.5/10)

The structural statistics in Table 3.1 separate the two sets and make the eventual classifier concrete. Planning-only modules synthesize to zero own cells at the median — their logic is their submodule wiring — and carry the longer, port-table-dominated specifica-

doi:10.6342/NTU202601985

tions. Direct-solvable modules are briefly specified, self-contained, and carry their modest logic entirely in-house (median 20 own cells, up to 640 for the S-boxes) with no dominant submodule structure. The two sets also differ in how they pass. Planning-solved tasks pass stably (7.7 of 10 samples on average, the interfaces being supplied rather than guessed), while direct-solved tasks pass by exploring the solution space (4.5 of 10). These are exactly the observable characteristics — delegation vs. own logic, interface-dominated vs. behavior-dominated specification — that the router's feature extractor (Section 3.5.3) is built to read from the specification alone.

We formalize the division as two task classes:

Class A (structural). Wrapper, integration, and memory-encapsulation modules. The module's own logic is thin; its difficulty is wiring externally defined submodule interfaces correctly. Class A tasks want the **planning flow**.

Class B (algorithmic). Self-contained datapath or control modules — cipher rounds, CRC, instruction decode, address generation. The difficulty is the logic itself, not integration. Class B tasks are the candidates for **direct** generation — not because the direct flow solves more of them, but because it solves the solvable ones at a tenth of the cost while the planning flow's overhead actively hurts.

For evaluation only, each task receives an oracle label computed from its golden implementation. The reference RTL is synthesized with Yosys, submodule instances are subtracted, and the task is labeled A if its own synthesized logic falls below a 250-cell threshold (or it is a recognized regular-array wrapper), otherwise B. Oracle labels use golden RTL that no generation-time component may see; they exist so that routing decisions can be audited against structure. Class membership alone does not decide which

doi:10.6342/NTU202601985

flow converts a task — most class-B tasks are solved by neither flow — so Section 5.7 scores routing correctness against empirical flow-solvability, whether each task solvable by exactly one flow was routed to that flow.

3.5.3 The Two-Tier Cascade

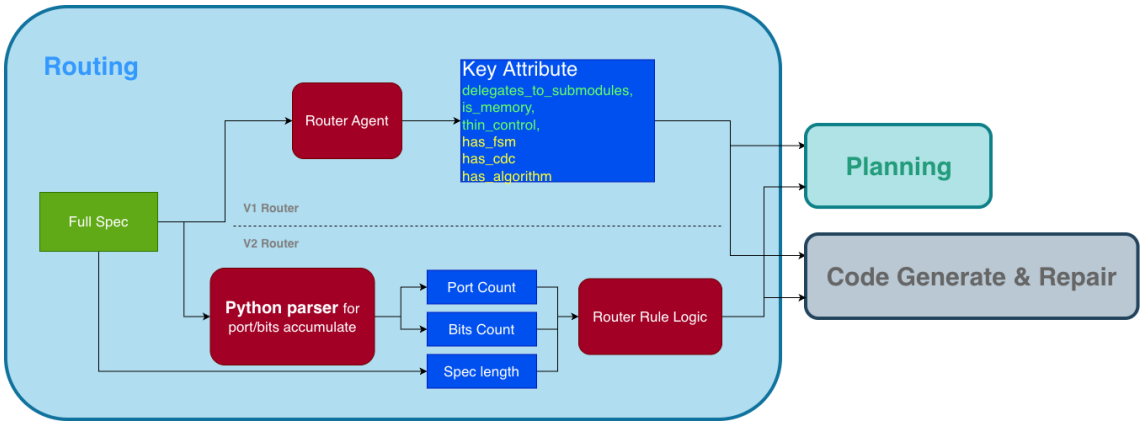
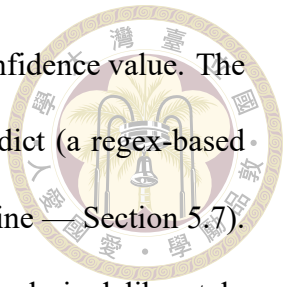


Figure 3.4: The spec pre-route in its two versions. Under the v1 rule the router agent reads the specification and emits the key routing attributes — delegation, memory, and thin-control signals for class A, FSM, CDC, and algorithm signals for class B — and the decision follows from these attributes alone, with an unconfident or undecided read falling through to the Tier-1 plan probe. The re-derived v2 rule gates deterministically instead. A small parser accumulates the port count from the specification's port tables and measures the specification length, and the rule logic combines these caps with the extractor's delegation judgment and own-state-bits estimate. Either way a task exits to the planning flow or to direct generation and repair.

The router is a cascade of two increasingly expensive tests; a task exits at the first tier that is confident. Figure 3.4 shows the first tier in both of its versions, the v1 read that routes from the extracted key attributes alone and the re-derived v2 rule that adds deterministic gates parsed from the specification itself.

Tier-0: spec pre-route. A feature extractor reads the specification and emits seven structured judgments: whether the module delegates to submodules, is a memory encapsulation, or is thin pass-through control (all class-A signals); whether it contains a finite-state machine, clock-domain crossing, or a datapath algorithm (class-B signals);

doi:10.6342/NTU202601985



and an estimate of the state bits the module itself must hold, plus a confidence value. The production extractor is a small LLM call that returns this JSON verdict (a regex-based keyword extractor over the same features serves as the ablation baseline — Section 5.7). The extractor prompt is reproduced in Appendix 6.4. The decision rule is deliberately transparent and conservative. The planning flow is the default. A task routes to direct only when the extractor reports no submodule delegation, its substantive logic is a self-contained algorithmic core, and it passes conservative gates on port count, specification length, and estimated own-state bits — the signature of the narrow computational leaves of Section 3.5.2. The gates are deterministic. A small parser counts the specification's port-table rows and measures its stripped length; only the own-state estimate comes from the extractor. This gated, planning-default form is the v2 rule of Figure 3.4. The earlier v1 rule decided from the extracted attributes alone, releasing any self-contained-core signal to direct, a polarity Section 5.7 traces to invalidated evidence and re-derives. When the extractor's confidence is below threshold, or its attributes leave the class undecided, the task falls through as uncertain.

Why an LLM extractor rather than parsing. The features themselves are simple enough that keyword matching looks tempting, but specification language is semantically misleading at exactly the decision boundary. A module can sound algorithmic while being structural. `e203_ifu_minidec` is described as performing "preliminary decoding," yet it is a thin wrapper that instantiates the real decoder. A keyword extractor reads "decoding", sees a self-contained core with no delegation vocabulary to contradict it, and releases the task to direct generation, where it has no access to the decoder interface it must wire. The reverse confusion also occurs. Specifications for genuinely algorithmic FIFOs and CRC engines spend most of their text on bus-facing port tables, so surface counting of inter-

doi:10.6342/NTU202601985

face vocabulary misreads them as structural. Deciding the class requires reading what the module does across the whole document — whether the described behavior is delegated or computed — which is a semantic judgment, not a lexical one. The whole-spec LLM read resolves both directions, and the ablation of Section 5.7 quantifies the gap to the keyword baseline.

What the specification cannot decide. Not every specification commits its module to either class. Across the 60 tasks, the extractor returns all-false features at full confidence for exactly five: `sd_bd` (a buffer-descriptor manager), `sd_data_serial_host` (the SD data-line serial engine), `e203_biu` (the bus interface unit), `e203_exu_alu_lsuagu` (the load/store address-generation unit sharing the ALU datapath), and `e203_exu_wbck` (the write-back arbiter). All of them are interface-arbitration modules that sit between protocols. They neither delegate to submodules (no class-A signal fires) nor implement a nameable algorithm or FSM (no class-B signal fires), but instead mediate, arbitrate, and translate between interfaces with modest own state. The specification genuinely does not commit the module to either class, and only the plan — which makes the delegation structure explicit — can. These are precisely the tasks the second tier exists for.

Tier-1: plan probe. Uncertain tasks run the planner once — the cheapest informative component of the planning flow — and the router probes the resulting structured plan's own natural-language requirement summary. The plan is a good routing feature for the same reason the condensation stage renders two views (Section 3.2). The planner view is built to surface each module's delegation and reuse structure while the generation view carries its full behavioral requirements. By the time a plan exists, characteristics that were ambiguous in the raw specification — does this module instantiate the work or perform

doi:10.6342/NTU202601985

it? — have been made explicit in the plan's own vocabulary of reuse decisions and requirement summaries. The probe simply reads that vocabulary back. Decisive delegation phrasing ("wraps", "encapsulates", "reuses existing", or instantiation language without any own-implementation verb) keeps the task in the planning flow, plan in hand; clear self-contained-algorithm phrasing routes it to direct, discarding the plan. The asymmetry of the default is intentional. The plan is already a sunk cost, so the probe bails to direct only when the module is clearly self-contained. A plan generated and then discarded is counted as a wasted plan — the cascade's overhead metric.

The direct flow. The direct arm is not a bare completion. It shares the generator's repair machinery. A directly generated candidate that fails the internal lint enters the same syntax repair loop, and (in arms where functional repair is enabled) the same functional repair loop, with the plan-dependent prompt sections simply absent. This matters for the routing economics. Direct generation costs roughly 10–12 k tokens per sample against the planning flow's 90–153 k, so every correctly-routed class-B task buys nearly an order of magnitude of compute reduction. Section 5.7 measures the realized economics.

Routing metrics. Section 5.7 evaluates routing arms on four axes: coverage (solved tasks and pass@k, against the planning flow's genuine coverage as the attainable ceiling); routing accuracy (against empirical flow-solvability, where routing an integration task to direct forfeits a would-be pass and routing an algorithmic task to the planning flow only forfeits the cost saving); overhead (wasted plans); and efficiency (estimated tokens and summed compute per solved task). The design goal, stated once and measured throughout, is to hold the attainable coverage at the lowest compute, with misroutes near zero.

doi:10.6342/NTU202601985

3.6 Summary

The methodology comes down to four commitments. Ground every decision. The planner works over the per-task catalog through its reuse tools, and every reuse decision it emits is also resolved against that catalog, so plans cite only modules the environment actually provides. Verify against the real environment. The inner lint compiles exactly what the benchmark will compile, making the two fatal contracts visible during repair rather than at scoring. The syntax-first-then-function gate ordering spends each feedback signal where it is most actionable. Reuse the model's own verified successes. Repair hints come from diffs that already fixed a structurally identical failure. Spend the expensive flow only where it converts. The cascade router reads each specification and gives integration-shaped tasks the full planning flow, while narrow self-contained ones take the cheap direct flow at a tenth of the cost. What each commitment buys is quantified in Chapter 5.

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

Chapter 4 Experimental Setup

This chapter fixes the experimental ground rules shared by every result in Chapter 5: the benchmark and its scoring semantics (Section 4.1), the models and serving infrastructure (Section 4.2), the metrics and the cost-accounting rules we commit to (Section 4.3), and a complete inventory of the experimental runs together with their configurations (Section 4.4). The inventory table doubles as the reproducibility anchor of this thesis. Every number reported later traces back to exactly one row of Table 4.2.

4.1 The RealBench Benchmark

All end-to-end evaluations use RealBench [10], a Verilog generation benchmark built from three real, open-source IP projects rather than from isolated textbook exercises. Each task provides a natural-language specification of one target module, the source files of the modules it may depend on, a golden reference implementation, and a self-checking testbench. The generator must produce the target module; the surrounding project files are supplied by the evaluation environment.

The benchmark distinguishes module-level tasks, which target one module inside an otherwise-provided project, from system-level tasks, which ask for an entire subsystem. This thesis reports on the **60 module-level tasks** summarized in Table 4.1; system-level

doi:10.6342/NTU202601985

Table 4.1: The 60 module-level RealBench tasks used in this thesis, by source project.

Family	Source project	Tasks	Character
AES	OpenCores AES-128 cipher core	6	self-contained datapath logic
SDC	OpenCores SD-card controller	14	bus-facing control logic
E203	HummingBirdv2 E203 RISC-V core	40	deep hierarchy, reuse-rich

tasks are excluded from all headline numbers for two reasons. Their specifications (up to ∼704 k characters for `e203_cpu_top`) exceed what any of our serving configurations can process without aggressive condensation, and a 60-task universe with three distinct families is already large enough to expose the phenomena this thesis studies. The three families span a useful difficulty gradient: AES modules are small and algorithmically self-contained, SDC modules implement Wishbone-facing control logic with wide interfaces, and the E203 family covers a complete RISC-V processor core whose modules instantiate one another — the setting in which IP reuse has the most to offer.

Scoring. RealBench scores with its own per-task `Makefile`, and its verdict is deliberately strict. A sample is counted as a syntax pass only if Verilator [28] compiles the full project — generated module, provided dependencies, reference model, and testbench together — with **no `%Error` and no `%Warning`** outside the Makefile's explicit waiver list; warnings are fatal. A sample is a functional pass ("pass" for short) only if it also compiles and the testbench simulation reports zero output mismatches against the golden reference. Two consequences of this design recur throughout the thesis. First, our syntax numbers are conservative relative to benchmarks that fail only on errors (Section 6.2). Second, the environment imposes two non-negotiable structural contracts on generated code. A dependency that the environment ships as a file must be instantiated, never re-declared (re-declaration is a fatal `MODDUP`), and the testbench's DUT instantiation fixes the exact port list (a missing port is a fatal `PINNOTFOUND`). Section 3.3 describes how the pipeline

doi:10.6342/NTU202601985

internalizes both contracts.

4.2 Models and Serving Infrastructure

All generation uses open-weight models served locally through vLLM [12] behind an OpenAI-compatible endpoint, on university/NCHC GPU nodes reached through SSH port forwarding. Three models appear in this thesis:

- **gpt-oss-20B** [23] — the workhorse model. All ablations (grounding, repair cache, repair budgets, routing) run on this model, so their comparisons are model-controlled.

- **gpt-oss-120B** [23] — used to test which findings survive a $6\times$ scale increase (Sections 5.8 and 5.7).

- **Qwen3-235B** [34] — used in supplementary repair-budget runs; it does not participate in headline comparisons.

One serving behavior shaped the implementation and is documented here because it is invisible in the results yet load-bearing for reproduction. The server caps completion length rather than rejecting a request when prompt and requested tokens together exceed the 131,072-token context window, but returns HTTP 400 when the prompt alone exceeds it. This is the condition behind the "not generated" bucket of Section 5.8 and one motivation for the specification condensation of Section 3.2.

doi:10.6342/NTU202601985

4.3 Metrics and Cost Accounting

Quality. For a run with n samples per task, we report the syntax rate and pass rate as per-sample means over all generated records, and pass@k per task with the standard unbiased estimator [5]

$$\text{pass@}k \;=\; \mathbb{E}_{\text{tasks}}\left[1 - \binom{n-c}{k} \Big/ \binom{n}{k}\right],$$

where c is the number of passing samples for the task, averaged over the 60 tasks. We also report solved tasks, the number of tasks with at least one passing sample. This equals $60 \cdot \text{pass@}n$ and is the quantity a practitioner who can afford n attempts actually cares about.

Cost. We use two cost measures. Estimated tokens are computed from prompt/response character counts with a calibrated 4.38-characters-per-token heuristic and summed per run; compute time (Σwall) is the sum of per-sample wall-clock seconds, which measures total machine work independently of the harness's $\sim$80-way request concurrency. Three accounting conventions apply throughout: (i) token totals from runs that were resumed after interruption are not cited as costs; (ii) token totals for the direct flow are reported as lower bounds; and (iii) wall-clock is compared only within a run — within-run sums and per-solved-task ratios — never across runs executed on different days on the shared server.

doi:10.6342/NTU202601985

4.4 Run Inventory

Table 4.2 lists every experimental run this thesis reports, the configuration that distinguishes it, and the section that consumes it. Short labels (left column) are used throughout Chapter 5. All runs share the same task discovery, per-task catalog construction, and RealBench evaluation harness, so rows differ only in the flow and the flagged configuration. Unless stated otherwise, sampled runs use temperature 0.2 (the runner default) and the two-stage planning flow of Chapter 3; "$6/4$" style entries abbreviate the syntax/functional repair-attempt budgets of Section 3.4.

Table 4.2: Inventory of all reported runs. Every number in Chapter 5 traces to one row. Directory names are given in Table 4.3.

Label	Model	$60 \times n$	Temp.	Distinguishing configuration
Baselines and headline pipeline				
DIRECT-T0	20B	$\times 1$	0	spec $\rightarrow$ model, no plan, no repair
PIPE-T0	20B	$\times 1$	0	grounded planner + generator, cache off
Repair-cache trio (identical except cache scope)				
CACHE-OFF	20B	$\times 20$	0.2	repair cache off
CACHE-TASK	20B	$\times 20$	0.2	repair cache, task scope
CACHE-RUN	20B	$\times 20$	0.2	repair cache, run scope
Model scale				
PIPE-120B	120B	$\times 10$	0.2	as CACHE-RUN, $6\times$ larger model
Functional repair and budgets				
FUNC-2/4	20B	$\times 10$	0.2	functional repair + spec slice, budgets $2/4$
FUNC-10/10	20B	$\times 10$	0.2	as above, budgets raised to $10/10$
FUNC-10/10-120B	120B	$\times 10$	0.2	as FUNC-10/10, $6\times$ larger model
Routing (Section 3.5 arms)				
HYBRID	20B	$\times(10{+}10)$	0.2	no router: both flows on every task
ROUTER-V1	20B	$\times 10$	0.2	full cascade (Tier-0 LLM + probe), $6/4$
ROUTER-V1-120B	120B	$\times 10$	0.2	as ROUTER-V1
ROUTER-V2	20B	$\times 10$	0.2	rule re-derived post-audit (planning-default, no probe), $6/4$, leak-filtered direct prompt

doi:10.6342/NTU202601985

Table 4.3: Mapping from run labels to repository directories under `runs/`.

Label	Directory
DIRECT-T0	`realbench_direct_model`
PIPE-T0	`agentic_plan_legacy_realbench_plan_hallu_fix_t0`
CACHE-OFF	`agentic_plan_legacy_realbench_oss20b_plan_hallu_no_rep_cache`
CACHE-TASK	`agentic_plan_legacy_realbench_oss20b_plan_hallu_task_rep_cache`
CACHE-RUN	`agentic_plan_legacy_realbench_oss20b_plan_hallu`
PIPE-120B	`agentic_plan_legacy_realbench_oss120b_plan_hallu_tool_call`
FUNC-2/4	`repair_spec_slice_oss20b_func_v2`
FUNC-10/10	`repair_spec_slice_oss20b_func_v2_time-err-off_10syn_10func`
FUNC-10/10-120B	`repair_spec_slice_oss120b_func_v2_time-err-off_10syn_10func`
HYBRID	`hybrid_oss20b_10syn_10func_rep_spec`
ROUTER-V1	`Full-2T_router_oss20B_sync6_func4`
ROUTER-V1-120B	`Full-2T_router_oss120B_sync6_func4`
ROUTER-V2	`Full-2T_router_oss20B_syn6_func4_s10_t02_v2`

Two rows deserve advance warning because they differ from their comparison partners in more than one variable. FUNC-10/10 changed both the repair budgets ($2/4 \rightarrow 10/10$) and a scoring waiver (`TIMESCALEMOD` warnings suppressed) relative to FUNC-2/4; Section 5.6 disentangles the two. And HYBRID predates the direct-flow repair loop that the router runs include, besides using the larger $10/10$ budgets. Section 5.7 therefore treats ROUTER-V1 $\rightarrow$ ROUTER-V2 as the only single-variable routing comparison and labels the comparison against HYBRID as directional. A third caution spans the whole inventory. The benchmark repaired its two S-box testbenches on June 25, 2026, so arms scored before the repair (DIRECT-T0, PIPE-T0, the CACHE trio, PIPE-120B, FUNC-2/4) and after it (FUNC-10/10, FUNC-10/10-120B, HYBRID, the router arms) are not comparable on `aes_sbox` and `aes_inv_sbox`. The cross-boundary comparisons in Chapter 5 exclude these two tasks (Section 5.9).

One scoring rule applies to every run with a direct flow. Samples whose generated RTL merely instantiates a leaked `ref_*` reference model are invalidated — counted as syntax and functional failures — before any rate is computed. Section 5.9 documents the leak, the detector, and the per-run impact.

doi:10.6342/NTU202601985

Chapter 5 Results and Analysis

This chapter evaluates the pipeline (Sections 5.1 through 5.6), the router (Section 5.7), and then turns the instruments on the failures themselves (Sections 5.8 and 5.9). All runs are identified by the labels of Table 4.2; all rates were recomputed directly from each run's per-sample records rather than transcribed from run logs. Every rate excludes ref-hack samples, generated candidates that pass by instantiating the benchmark's own reference model through a prompt-side leak. The audit of Section 5.9 details and quantifies this benchmark-gaming pattern.

5.1 Does Planning and Reuse Beat the Model Alone?

The cleanest head-to-head is deterministic. One greedy sample per task, identical harness, PIPE-T0 versus DIRECT-T0 (Table 5.1, Figure 5.1).

Table 5.1: Grounded pipeline vs. pure model on the 60 module tasks, temperature 0, one sample per task. Like every number in this chapter, rates exclude ref-hack samples (Section 5.9); the exclusion costs DIRECT-T0 one syntax and one pass.

Arm	Syntax	Pass
PIPE-T0 (grounded planner + generator)	**37/60 = 61.7 %**	**15/60 = 25.0 %**
DIRECT-T0 (pure model)	15/60 = 25.0 %	5/60 = 8.3 %

The pipeline delivers $2.5\times$ the syntax rate and $3.0\times$ the pass rate of the same model prompted directly. The family breakdown locates nearly the entire effect. On AES —

doi:10.6342/NTU202601985

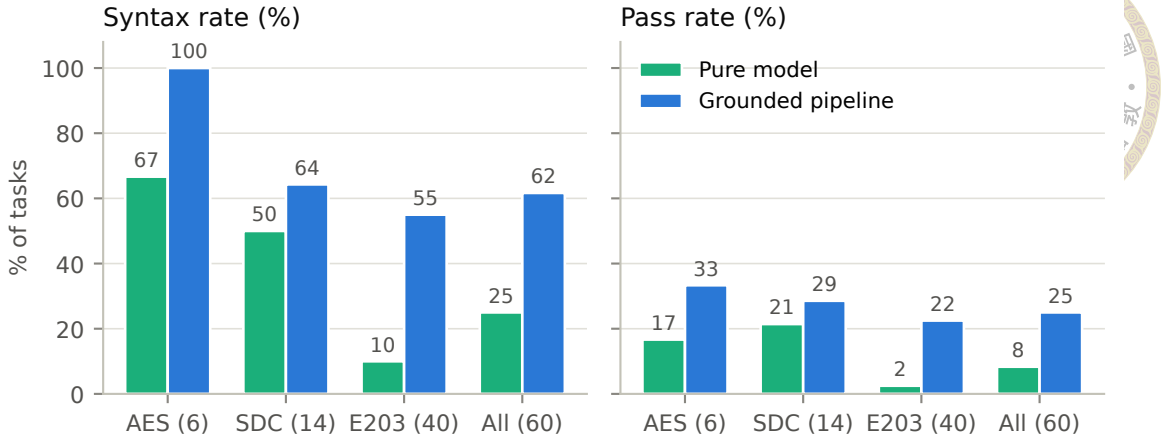


Figure 5.1: Syntax and pass rates by family, pure model vs. grounded pipeline (temperature 0, one sample). The aggregate gain is carried almost entirely by the reuse-rich E203 family.

small, self-contained, nothing to reuse — the model alone compiles two thirds (66.7 %) and the pipeline's pass gain is one task (16.7 % → 33.3 % of six). On SDC the pipeline helps modestly (pass 21.4 % → 28.6 %). On E203, the family whose modules instantiate one another, the pipeline is decisive, with syntax 10.0 % → 55.0 % and pass 2.5 % → 22.5 %. The conclusion we draw — and will lean on again when routing — is that the pipeline's value is proportional to how much legitimate IP reuse a task affords. Where there is nothing to ground, planning is overhead; where the task is an exercise in wiring real submodules, injecting their true interfaces raises the syntax rate more than fivefold.

5.2 The Grounding Audit: Correct, Auditable Plans

Grounding-by-construction (Section 3.2) does exactly what it was built to do. In PIPE-T0, the 60 plans contain 109 reuse decisions; 105 already name an exact catalog id, 4 near misses are remapped, and 0 hallucinations survive (Figure 5.2). At twenty samples (CACHE-RUN), the same mechanisms process 1,934 decisions — 1,834 exact, 100 remapped, 0 leaked. Before the fix, plans routinely carried invented module names; after

doi:10.6342/NTU202601985

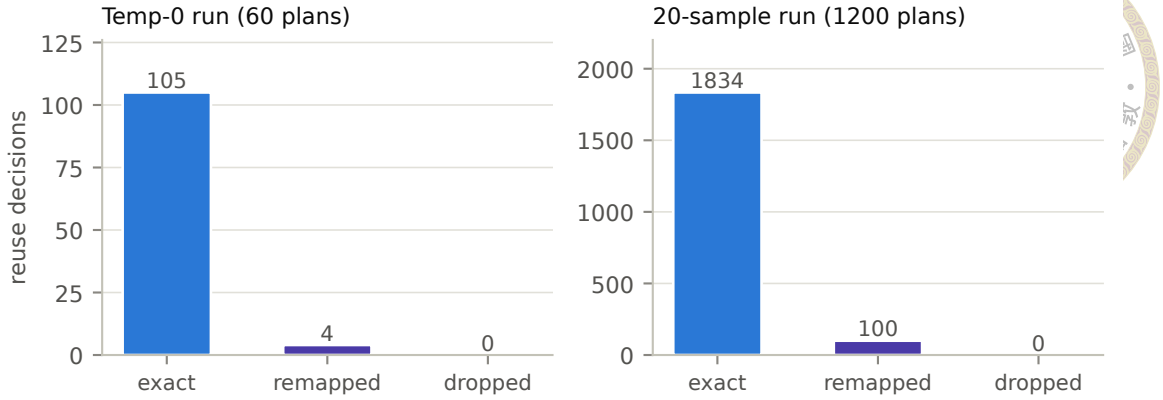


Figure 5.2: Grounding outcomes for every reuse decision in two runs. Near-miss names are remapped onto real catalog ids; nothing hallucinated survives into the plan.

it, every reuse decision in every plan resolves to a real, provided module. The planner's reuse matrix, previously empty because decisions hid under a dozen ad-hoc JSON keys, also became fully populated and auditable.

The end-to-end pass rate, meanwhile, stays level; pass@1 moved from 19.6 % (ungrounded twin run) to 20.2 % — +0.6 percentage points, within noise. We report this measurement deliberately, because it locates the mechanism's value precisely. The generator was already robust to imperfect names, discarding them and re-deriving reuse from the real dependency set, so grounding bought plan correctness and auditability. Every reuse decision now resolves to a real module and can be inspected, which is what the router's plan probe (Section 3.5.3) and the reuse-matrix analyses build on. Plan quality was not the binding constraint on yield; Sections 5.8 and 5.9 show what is.

The closed vocabulary earns its keep a second time at the generation stage, where the cost of leaving it open is directly measurable. Both flows tell the model which modules the verification environment already provides, so that it instantiates rather than redeclares them. The planning-flow runner curates that list and strips every `ref_*`, testbench, and stimulus name before the prompt is built. The direct flow's original prompt passed the

doi:10.6342/NTU202601985

list through unfiltered and named `ref_<task>`, the benchmark's own golden reference model; a generated top module that merely instantiates it passes by construction, and the models found the exploit (the audit in Section 5.9 tells that story). Table 5.2 counts the resulting ref-hack samples in two runs with the leaky direct prompt, a matched planning-flow run on the same tasks, and the router run whose direct prompt carries the filtered replacement.

Table 5.2: Ref-hack samples by arm and flow. A sample is a hack if any line of its generated RTL instantiates a `ref_*` module. Passing hacks are included in raw pass counts; every rate elsewhere in this thesis excludes them.

Arm	Flow	Samples	Hacks	Passing
DIRECT-T0	direct	60	17	1
DIRECT	direct	600	186	48
PLANNING	planning	600	0	0
ROUTER-V2	direct (filtered)	110	0	0
	planning	490	0	0

The asymmetry is stark. With the reference named in the prompt, the model hacked 186 of DIRECT's 600 samples and 17 of DIRECT-T0's 60, and the hacks that compiled account for half of DIRECT's raw passes (48 of 98) and one of DIRECT-T0's six. With the vocabulary curated, the exploit disappears; PLANNING's 600 samples on the same tasks contain none, and ROUTER-V2, whose direct prompt adopts the same filter, produced zero in either flow. One caveat keeps the claim honest. Across all 7,760 planning-flow samples in the run inventory the screen finds no hack; the one hack a curated prompt ever produced is a probe-routed ROUTER-V1 sample of `e203_exu_alu_lsuagu` (one passing sample), released to direct generation after its plan was discarded. Nothing in that sample's recorded plan, repair events, or Verilator diagnostics names a `ref_*` module — the filter held, and the model reproduced the benchmark's naming convention from prior knowledge alone. Closing the vocabulary removes the invitation but cannot erase a guessable name, which is why hack screening remains in the scoring path independent of prompt hygiene. The

doi:10.6342/NTU202601985

protection here is the same one the grounding audit measures on plans. What the prompt is allowed to say bounds both what the model can hallucinate and what it can exploit.

5.3 Sampling Structure: pass@1 to pass@10

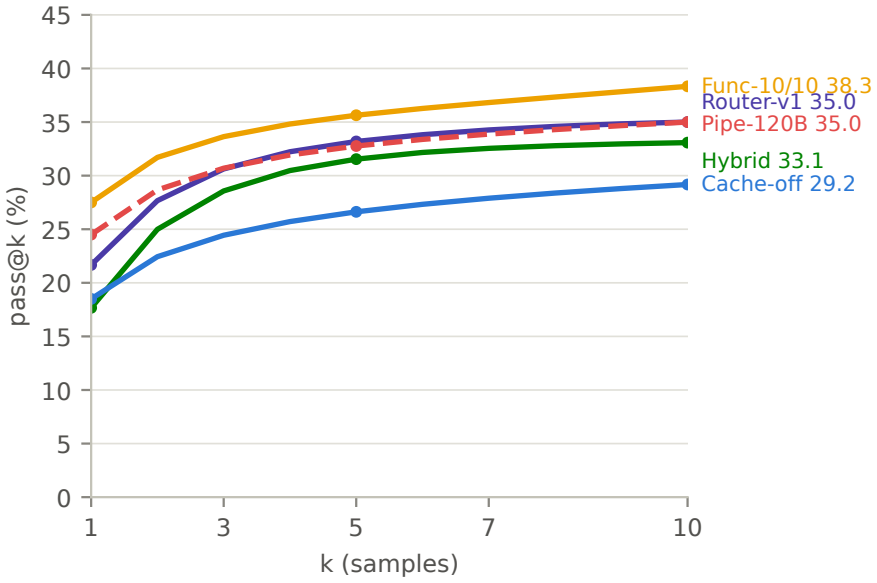


Figure 5.3: Unbiased pass@k for $k = 1$–10 across five arms on one axis. Normalizing at $k \leq 10$ lets every 60×10 run be read together with the 60×20 runs (whose curves the estimator subsamples). System design separates the curves at least as much as model scale does; the FUNC-10/10-vs-PIPE-120B margin partly reflects the June-25 S-box fix boundary (see text).

Because most arms sample ten times per task, drawing the sampling structure from pass@1 to pass@10 puts every run on one common axis (Figure 5.3); the 60×20 arms enter through the same unbiased estimator. Read together, the curves expose three facts that single-run views hide. First, diversity helps every system by a similar factor and saturates fast. From k=1 to k=10 each arm gains 1.4–1.9× (CACHE-OFF 18.5 %→29.2 %, FUNC-10/10 27.5 %→38.3 %, HYBRID 17.7 %→33.1 %, ROUTER-V1 21.7 %→35.0 %), with most of each gain realized by k=5. Extending CACHE-OFF to its full twenty samples adds only +2.5 pp more (31.7 % at k=20), so samples beyond ten buy little anywhere. Second, the

doi:10.6342/NTU202601985

curves rarely cross. Arms are ordered at $k{=}10$ almost as at $k{=}1$, and system differences matter at least as much as scale differences. The 20B FUNC-10/10 curve runs above the 120B planning-flow curve (PIPE-120B, 24.5 %→35.0 %) at every k and tops the chart. Part of that margin, however, is a scoring artifact rather than system value. PIPE-120B was scored before the June-25 S-box testbench repair and FUNC-10/10 after it (Section 5.9), so the two S-box tasks were unwinnable for the 120B arm. Excluding both tasks, the 20B system still edges the $6\times$ larger model at pass@1 (26.4 % vs. 25.3 %) and the two arms tie exactly at pass@10 (36.2 %). On equal terms, deep repair around a 20B model matches the 120B planning flow but does not beat it. The confound-free version of the scale question is FUNC-10/10 against FUNC-10/10-120B, the same deep-repair system on the larger weights with both arms scored after the S-box fix (Table 5.3). There every rate moves up together — syntax@1 81.5 % to 89.8 %, pass@1 27.5 % to 33.2 %, pass@10 38.3 % to 40.0 % — and the 120B arm's 24 solved tasks are a strict superset of the 20B arm's 23. So system and scale compose, but most of what scale buys is pass@1 precision; the $k{=}1{\to}10$ gain factor falls to $1.2\times$, below every 20B arm's. Third, the task universe splits sharply beneath every curve. On CACHE-OFF, 19 of 60 tasks are solved at least once in twenty tries and the remaining 41 never pass at any k; the sampling headroom lives entirely inside the solvable set. One decoding note carries over from the deterministic arms. Greedy decoding beats the sampled per-sample mean (PIPE-T0 25.0 % against 18.5–20.2 %), the standard precision-versus-diversity trade; temperature is what buys the pass@10 headroom.

Against the published RealBench numbers. The same @1/@5 axis lets our arms be read directly against the RealBench study's own evaluation of ten frontier and Verilog-specialized models [10] (Table 5.3). The published protocol matches ours closely, with the same 60 module tasks, the same Makefile verdict, the same temperature (0.2) and un-

doi:10.6342/NTU202601985

biased estimator, and @1/@5 estimated from twenty samples against our ten. It prompts every model directly, however, so the comparison isolates what the system around the model is worth. The strongest published rates are func@1 13.5 % (GPT-4o-V) and func@5 17.3 % (DeepSeek-V3-671B). DIRECT-T0, our 20B model under the same direct prompting, lands inside that frontier band (8.3 %). With the grounded planning flow and repair loops around it, the identical weights roughly double the best published func@1 and func@5 (FUNC-10/10: 27.5 % and 35.6 %) and close to triple the best published syntax@1 (81.5 % vs. 29.3 %). Carrying the same system to the open 120B weights (FUNC-10/10-120B) stretches the margin further, to $2.5\times$ the best published func@1 (33.2 %) and $3.1\times$ the best published syntax@1 (89.8 %). The Verilog-specialized fine-tunes fare worst of all on this benchmark (CodeV-QW-7B func@1 0.3 %), reinforcing this section's reading that system design separates RealBench performance more than model identity or scale does. Two caveats bound the comparison. First, the published runs predate the June-2026 AES S-box testbench fix (Section 6.2), though excluding both S-box tasks moves none of our rates by more than 2.3 pp. Second, RealBench also reports formal@k (equivalence checking against the golden model), which our simulation-based harness does not measure, so the table compares the two shared metrics only.

5.4 Repair-Cache Ablation: Density, Not Solvability

The cache trio (CACHE-OFF/-TASK/-RUN) differs in exactly one flag, giving the thesis's cleanest three-arm ablation (Table 5.4, Figure 5.4).

The cache does what a syntax-hint mechanism can do. Verified fix hints lift the syntax rate monotonically ($+4.9$ pp at task scope, $+7.5$ pp at run scope) and pull per-sample pass

doi:10.6342/NTU202601985

Table 5.3: Module-level RealBench results published in the RealBench study [10] (direct prompting, temperature 0.2, @1/@5 estimated from 20 samples per task; o1-preview sampled once) against this thesis's arms on the same 60 tasks and the same Makefile verdict (temperature 0.2, 10 samples; the -T0 arms are greedy single samples, so their @5 is undefined). Our rates exclude ref-hack samples (Section 5.9); excluding the two post-fix-boundary S-box tasks as well would shift no rate by more than 2.3 pp.

	Syntax (%)		Functional (%)	
Model / arm	@1	@5	@1	@5
Single LLMs, direct prompting; published rows from [10]				
GPT-4-Turbo	27.0	36.2	9.4	13.8
o1-preview	28.3	—	13.3	—
Llama-3.1-8B	10.3	21.7	2.9	6.2
Llama-3.1-405B	6.6	15.7	2.8	7.6
DeepSeek-V3-671B	24.9	32.0	12.8	17.3
DeepSeek-R1-671B	13.4	30.4	8.0	16.2
RTLCoder-DS-6.7B	10.9	19.0	3.4	8.5
CodeV-QW-7B	3.6	7.4	0.3	1.0
GPT-4o	24.5	29.7	10.9	14.0
GPT-4o-V	29.3	38.1	13.5	16.2
gpt-oss-20B (DIRECT-T0, ours)	25.0	—	8.3	—
This thesis: system around the same open weights				
PIPE-T0 (20B, grounded planning)	61.7	—	25.0	—
FUNC-10/10 (20B, planning + repair)	81.5	90.7	27.5	35.6
FUNC-10/10-120B (120B, same system)	**89.8**	**93.3**	**33.2**	**39.5**

Table 5.4: Repair-cache scope ablation (60×20, gpt-oss-20B, identical configuration otherwise).

Cache scope	Syntax	pass@1	pass@20	mean repair attempts
off	59.4 %	18.5 %	**31.7 %**	1.12
task	64.3 %	19.6 %	**31.7 %**	1.32
run	66.9 %	20.2 %	**31.7 %**	1.61

doi:10.6342/NTU202601985

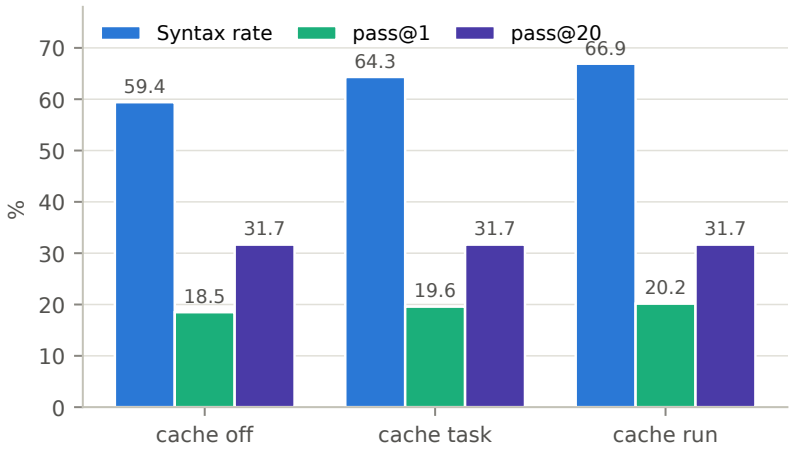


Figure 5.4: The cache lifts syntax monotonically and drags pass@1 with it, but pass@20 does not move at all.

up with it, at the cost of more repair turns taken. What it cannot do is change which tasks are solvable; pass@20 is identical to the decimal across all three scopes. The cache shifts the density of passing samples inside already-solvable tasks; functional bugs — the actual frontier — are immune to syntax-diff hints.

One qualification: these are three independently sampled runs, so the monotone ordering of syntax and pass together is suggestive rather than proof; individual deltas of 1–2 pp are within sampling noise. Run scope's further edge over task scope (+2.6 pp syntax, +0.6 pp pass@1) sits inside that band, so **task scope is the default later runs adopt**. Token totals across this trio are not comparable (the task/run arms were resumed mid-run; Section 6.2); the one clean cost figure is CACHE-OFF's 46.3 M estimated tokens for 1,200 samples.

The cache in action, across tasks. A single run-scope event shows what the aggregate numbers only imply, and it shows the transfer working between two different tasks. In PIPE-120B, the generator for `e203_itcm_ram` instantiated the provided `sirv_gnrl_ram` correctly but dropped the leading backtick on every macro argument,

doi:10.6342/NTU202601985

so E203_ITCM_RAM_DP and the port-width macros reached Verilator as bare, undeclared identifiers rather than expanded constants. The compile died with eight errors, the loudest being that a defparam value would not reduce to a constant (Listing 5.1). On the first repair attempt the run-scope cache returned a stored fix at cosine similarity 0.96 and injected it as an advisory hint. The fix came from e203_dtcm_ram, repaired earlier in the same run, where the identical omission on its own RAM instance had already been corrected once and verified.

```
// before repair -- macros lack the backtick, so Verilator
// rejects the defparam value as non-constant
sirv_gnrl_ram #(
    .DP (E203_ITCM_RAM_DP),
    .DW (E203_ITCM_RAM_DW),
    .MW (E203_ITCM_RAM_MW),
    .AW (E203_ITCM_RAM_AW)
) u_itcm_ram ( ... );

// after repair -- backticks restored, macros expand to constants
sirv_gnrl_ram #(
    .DP ( `E203_ITCM_RAM_DP ),
    .DW ( `E203_ITCM_RAM_DW ),
    .MW ( `E203_ITCM_RAM_MW ),
    .AW ( `E203_ITCM_RAM_AW )
) u_itcm_ram ( ... );
```

Listing 5.1: The missing-backtick fault in e203_itcm_ram and the cached repair transferred from e203_dtcm_ram. Macro arguments without the leading backtick read as non-constant identifiers; restoring it lets them expand to their defined depths and widths.

The transfer is genuinely cross-task, and the two grounding mechanisms divide the work cleanly. e203_itcm_ram was the first of its samples to fail, so no fix from its own

doi:10.6342/NTU202601985

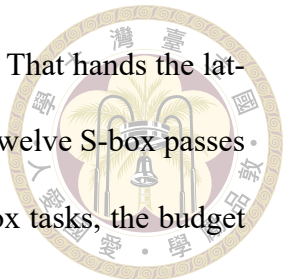
task existed yet, and the only match in the cache carried the other task's macro names — which is why the similarity is 0.96 rather than 1.0. A cache that pasted its stored diff verbatim would have written `E203_DTCM_RAM_DP` into an ITCM module. It does not, because the hint is advisory and the repair prompt also carries the specification's interface slice with this module's own macro names and widths. The model reapplies the backtick pattern the hint demonstrates while filling in the values the slice supplies, and the candidate compiles and passes after one repair turn. The hint supplies the pattern, and the slice supplies the values.

This is the density mechanism of Table 5.4 at the scale of a single sample. A fix the model verified once is replayed to clear a structurally identical failure in one turn instead of rediscovering it, which lifts the syntax rate and the passing-sample density without touching solvability — the `dtcm` bug was already fixable, and the cache only makes the `itcm` instance of it cheaper to reach.

5.5 Functional Repair: a Separate, Oracle-Fed Arm

The functional repair loop and specification slicing (Sections 3.4.2–3.4.3) are evaluated as their own arm because their feedback signal is the scoring testbench itself. Raising the budgets from $2/4$ (FUNC-2/4) to $10/10$ (FUNC-10/10) lifts syntax from 60.3 % to 81.5 % and pass from 20.2 % to 27.5 %, but at $2.7\times$ the cost (33.5 M $\rightarrow$ 89.8 M tokens). Two confounds must be subtracted before the budgets get the credit. The arms differ in one scoring waiver (`TIMESCALEMOD` warnings suppressed in FUNC-10/10), so the syntax delta slightly overstates the budget effect alone. They also straddle the June-25 S-box testbench repair (Section 5.9). FUNC-2/4 was scored on the broken benches — its S-box failures carry the

doi:10.6342/NTU202601985



race's 766-mismatch signature — and FUNC-10/10 on the fixed ones. That hands the latter arm +10 of its +44 additional passing samples for free (ten of its twelve S-box passes compile and pass with no repair of either kind). Excluding both S-box tasks, the budget lift is 20.5 % to 26.4 % pass, +34 samples. Attributing those is sobering. +27 are candidates that, thanks to deeper syntax repair, now compile and then pass the testbench on their first functional check; only +7 net passes are attributable to the functional repair loop itself. The clear winner inside this arm is therefore deeper syntax repair; the next section calibrates where each loop's budget is best spent.

The same configuration on gpt-oss-120B (FUNC-10/10-120B) shifts that attribution somewhat. Of its 199 passing samples, 173 pass their first functional check and 26 are functional-repair rescues, a third more than the 20B arm's 19. Where the rescues land matters more than their count. Two tasks enter the arm's solved set only through the loop, with three rescued samples each and none born passing — `e203_exu_commit`, which no 20B arm solves, and `e203_exu_decode`, the decoder whose compiling samples every pre-repair arm got wholly wrong and which both FUNC-10/10 arms convert only this way (one rescue at 20B, three at 120B). The stronger model extracts more from the same mismatch signal, so the loop's value grows with scale rather than shrinking.

5.6 Repair-Budget Calibration: Fix Fast or Never

Both repair phases share one shape — fix fast or never — but with very different economics (Figure 5.5).

doi:10.6342/NTU202601985

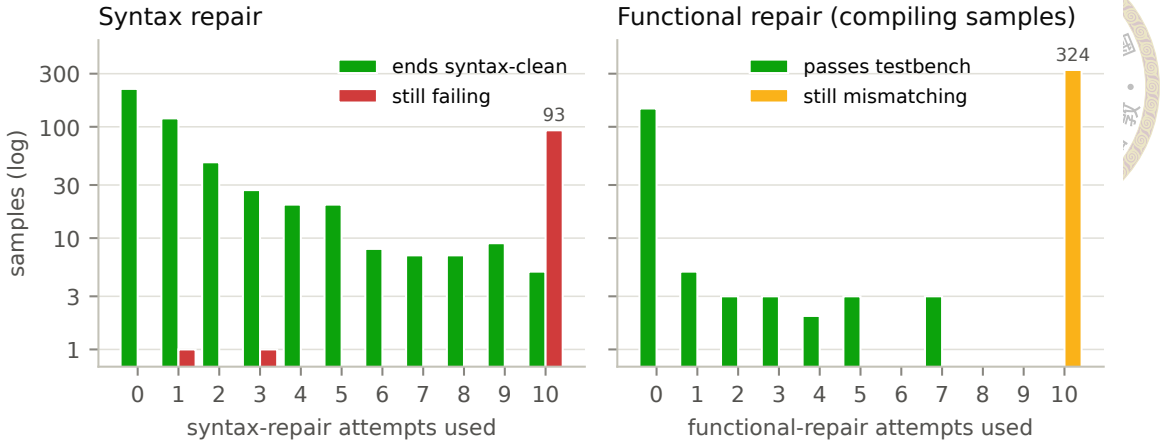
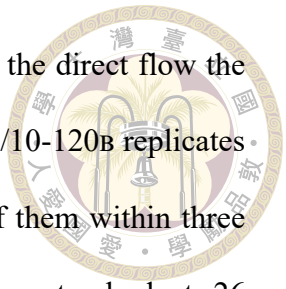


Figure 5.5: FUNC-10/10: outcomes by repair attempts consumed (log-scale counts, so the single-digit late-attempt buckets stay visible; empty buckets have no bar). Left: samples by syntax-repair attempts; nearly every bucket short of the cap ends syntax-clean, while the cap bucket (10) is 95 % failures. Right: compiling samples by functional-repair attempts; every rescue lands by attempt 7, and 326 samples burn the full budget converting nothing.

Syntax repair earns its budget. In FUNC-10/10, 219 samples compile without repair; repair rescues another 265 by attempt 9, with meaningful yield still arriving at attempts 4–6. Past that point conversions become rare (102 samples reach attempt 10; 5 succeed). The independent ROUTER-V1 run confirms the shape at budget $6/4$ — 153 born-clean, 228 repair-rescued with a real tail across all six attempts (73/53/43/23/22/14), and the rest exhausted or invalidated as ref-hacks. A syntax budget of about six captures nearly all attainable value.

Functional repair converges even faster. Of the compiling samples in FUNC-10/10 that enter the functional phase failing, every rescue the loop achieves — 19 of them — lands by attempt 7, and the large remainder (326 samples) runs the full depth without converting. Since each attempt is a full compile-plus-simulation (up to 300 s) plus an LLM call, a depth-ten functional budget spends $3{,}260$ of $3{,}324$ attempts past the point where rescues occur. That is exactly where the arm's $2.7\times$ cost multiple lives, and exactly the compute a tighter cap recovers. ROUTER-V1 confirms the same convergence at scale

doi:10.6342/NTU202601985



(13 rescues, all early, against 251 samples that run the depth out; on the direct flow the planning-tuned prompts convert 2 of 286 genuine samples). FUNC-10/10-120B replicates both distributions on the larger model — 213 syntax rescues, 187 of them within three attempts and the last at attempt nine, while 55 samples burn the full syntax budget; 26 functional rescues, all by attempt eight, while 340 compiling samples run the functional depth out, so $3{,}400$ of its $3{,}463$ functional attempts convert nothing. At 120B a functional cap of four would forgo three of the 26 late rescues but no solved task. The calibration adopted for later runs follows directly — syntax cap ≈ 6, functional cap ≤ 4, plus a no-progress early exit (stop when the mismatch count stops dropping). These budgets keep essentially all of the rescues at a fraction of the compute. The mismatch signal is coarse, and Section 5.9 shows it can even be misleading; why it supports only early, easy fixes is taken up there.

5.7 Routing: Coverage Held under a Re-Derived Rule

Table 5.5: Routing arms, ref-hack samples excluded (Section 5.9). All arms run gpt-oss-20B except FUNC-10/10-120B, included for scale reference; its Σwall comes from a different serving deployment and is not comparable to the 20B rows. syntax@1 is the share of samples that compile; both rates in percent. Solved = tasks (of 60) with ≥ 1 passing sample. Σwall sums per-sample wall-clock. HYBRID and the FUNC arms use budgets $10/10$, and HYBRID predates direct-flow repair; the single-variable comparison is ROUTER-V1→ROUTER-V2 (replaces the fitted rule with the re-derived one).

Arm	Samples	syntax@1	pass@10	Solved	Σwall (h)	Mtok/solved
HYBRID (both flows)	600+600	48.9	33.1	20	423	4.87
FUNC-10/10 (planning)	600	81.5	38.3	23	402	3.90
FUNC-10/10-120B (planning)	600	**89.8**	**40.0**	**24**	125	3.86
ROUTER-V1 (fitted rule)	600	63.5	35.0	21	130	1.35
ROUTER-V2 (re-derived)	600	76.5	35.0	21	262	2.49

Table 5.5 and Figure 5.6 give the routing result. The benchmark-gaming audit (Section 5.9) does more to this section than rescale its numbers. It invalidates the solution set the routing rule was fitted on. The v1 cascade's polarity — algorithmic signals route to

doi:10.6342/NTU202601985

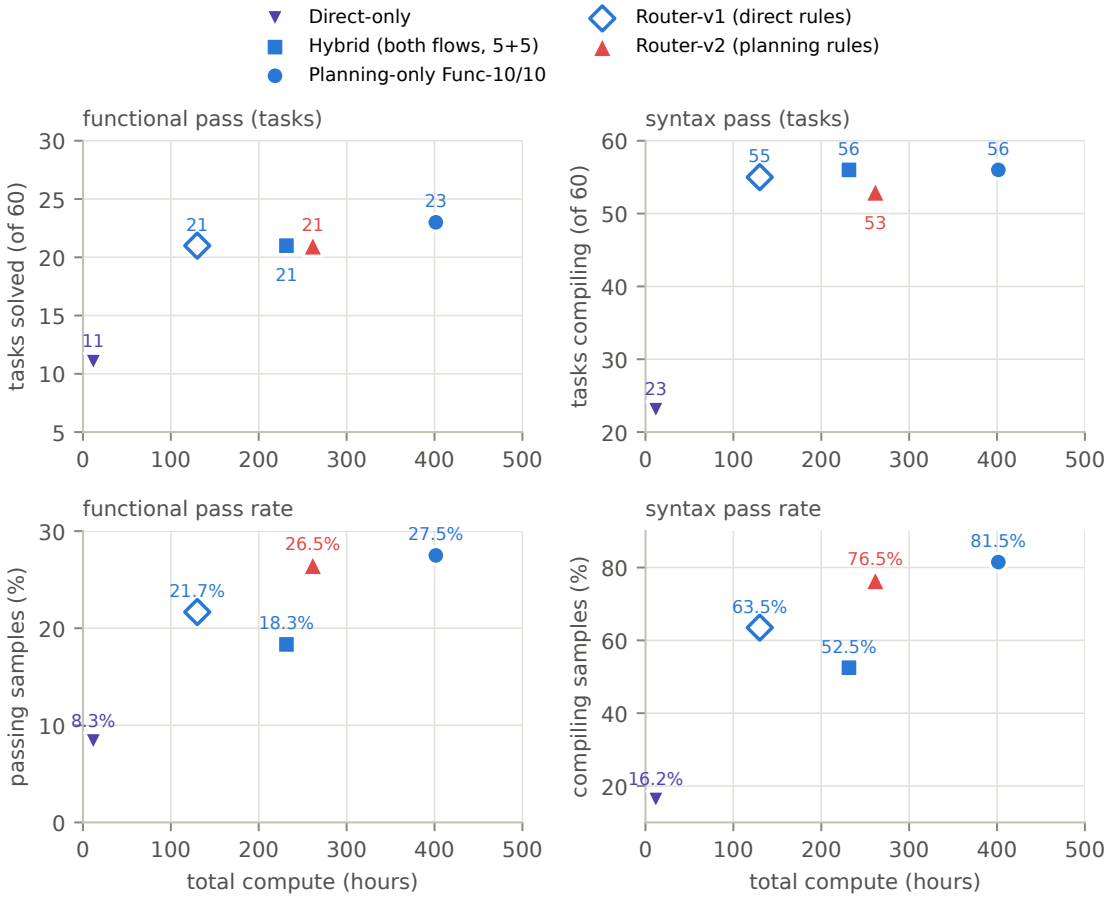


Figure 5.6: The gpt-oss-20B arms against total compute. The legend identifies each arm's marker; each point is labeled with its value. The HYBRID point is a 5+5 rerun of both flows, so every plotted arm carries 60×10 samples; the 10+10 run behind Table 5.5's HYBRID row solves 20 tasks at 423 hours on twice the sample budget. The DIRECT-ONLY point is that 10+10 run's direct flow taken alone, 600 unrepaired direct samples anchoring the low-compute end at 12 hours; it is plotted for reference and is not a routing arm. The v1 cascade is drawn hollow because its rule was fitted to the pre-audit solution set and its 130-hour saving is coupled to the invalidated categorization (see text). Top row, task counts. In tasks solved (left), the direct flow alone reaches 11 tasks, the 21-task arms sit on one horizontal line, the re-derived ROUTER-V2 solves the same 21 tasks at 262 hours, and the planning-only 10/10 arm buys two more tasks at the highest cost; in tasks compiling (right), syntax reach is nearly flat at 53–56 tasks against the direct flow's 23. Bottom row, per-sample rates. The routed arms tie on task counts but separate cleanly on functional (26.5 % vs. 21.7 %) and syntax (76.5 % vs. 63.5 %) pass rates.

doi:10.6342/NTU202601985

direct — was derived from records in which the direct flow appeared to solve FSM- and algorithm-heavy tasks it was in fact "solving" by instantiating the benchmark's leaked reference models. We therefore evaluate two versions of the decision rule on the same cascade infrastructure: ROUTER-V1, which runs that fitted polarity, and ROUTER-V2, which runs the designed rule of Section 3.5.3. The routing claim this thesis stands on is ROUTER-V2's row.

The v1 cascade's compute win was fitted to the wrong evidence. ROUTER-V1 places Tier-0 pre-routing in front of the probe and is the cheapest arm in Table 5.5 at 130 hours, because 342 of 600 samples run on the direct flow at roughly a tenth of a planning-flow sample's token cost. Before the audit, the same records read far more favorably still. The cascade appeared to solve 34 tasks at pass@10 56.7 %, thirteen more than survive the cleaning. The audit traced all thirteen — and, on inspection, the compute saving's origin — to the same place. The apparent direct-flow strength on the AES tops, SD-card engines, and E203 decode/dispatch tasks was the hack pattern; it is exactly that apparent strength the v1 polarity encodes. With hacks excluded, the planning flow matches or exceeds the direct flow on 58 of 60 tasks, and v1's direct arm converts only 14.3 % of its samples cleanly. The 130-hour figure is the cost of sending 32 tasks — the expensive giant integrators among them — to a flow that mostly could not solve them, while the hacks hid the damage. The coverage survives the cleaning (21 of 60); the categorization behind the economics does not. We report this in full because the audit that produced it is part of the contribution (Section 5.9).

Figure 5.7 traces how ROUTER-V1's $60 \times 10 = 600$ samples move through the cascade's tiers, together with the volume the repair loops handle in each flow. Under the v1

doi:10.6342/NTU202601985

polarity, Tier-0 routes 55 of 60 tasks from the specification alone (32 tasks to direct, 23 to the planning flow); the remaining 5 tasks' samples pay one planner call each, after which the probe keeps 28 samples in the planning flow and releases 22 to direct. Of the resulting 342 direct and 258 planning-flow samples, the syntax repair loop delivers 205 and 176 genuinely compiling candidates respectively (a further 55 direct-flow "compiles" were ref-hacks and are invalidated; Section 5.9). The direct flow compiles at 59.9 % with most samples clean on arrival, while the planning flow leans on the loop harder, as expected for the giant integrators routed to it. The functional stage converts 130 samples into passes across both flows.

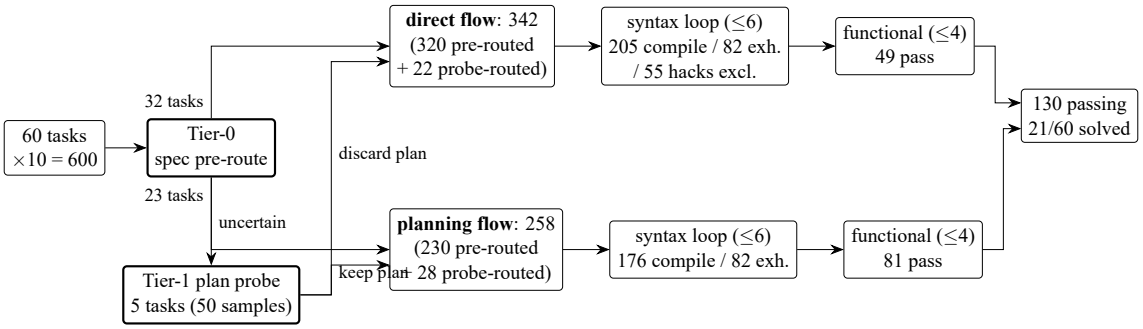


Figure 5.7: How ROUTER-V1's 600 samples route through the tiers and the shared repair loops under the v1 polarity (gpt-oss-20B, budgets $6/4$; ref-hack samples excluded per Section 5.9). Tier-0 decides 55 of 60 tasks from the specification; the plan probe resolves the 5 uncertain tasks per-sample. Both flows run the same syntax and functional repair loops; counts give each stage's outcomes.

The re-derived rule. ROUTER-V2 keeps the Tier-0 feature extractor and runs the rule of Section 3.5.3 — planning flow by default, with direct generation only for modules that delegate to no submodule and pass the conservative gates on port count, specification length, and estimated own-state bits (at 20B: ports ≤ 30, specification ≤ 6 k characters, own state ≤ 20 bits, relaxed to ≤ 64 bits when the extractor reports a self-contained datapath algorithm). These are the narrow, self-contained computational leaves (S-boxes, CRC engines, FIFO and clock utilities), the only tasks the direct flow genuinely solves

doi:10.6342/NTU202601985

after cleaning. Under this rule no task is ambiguous enough to reach the plan probe. In all, 11 of 60 tasks route direct (110 samples), 49 to the planning flow, and the wasted-plan overhead is zero (v1: 22 discarded plans). The run also postdates the prompt-side leak fix, so it is the one routed arm whose raw records contain no ref-hack sample at all.

Head-to-head: same coverage, better samples. Cleaned, ROUTER-V1 and ROUTER-V2 solve the same 21 of 60 tasks (20 in common; the single-task difference in each direction is a ∼1-in-50-density task, inside sampling noise). Per sample, v2 is strictly better. Its pass@1 is 26.5 % against 21.7 %, its syntax 76.5 % against 63.5 % (Table 5.5), and its direct arm passes 49.1 % of its samples cleanly against v1's 14.3 %. The carve-out sends the direct flow only what it can genuinely do. Scored against empirical flow-solvability pooled over all seven cleaned runs (13 tasks solvable by both flows, 11 planning-only, 4 direct-only, 32 by neither), v1 routes 13 of the 15 single-flow tasks to their solving flow and v2 routes 12. But v1's two losses are ∼1-in-10-density planning-only tasks, one of which (`e203_exu_oitf`) v2 recovers, while v2's three losses are ∼2 %-density direct-only tasks that v1 also failed to convert despite routing them direct. Realized coverage is identical. (Routing is not scored against a class-label confusion matrix. As Section 3.5.2 states, class membership alone does not decide which flow converts a task — only 4 of the 28 class-B tasks are empirically direct-only.)

The economics, corrected. ROUTER-V2's 262 hours is, in effect, the cost of running every task through the planning flow. The reason is structural. The 11 tasks v2 routes direct are exactly the tasks that are also cheapest on the planning flow; run through it, they cost 0.47 M tokens and 3.6 hours, about 1 % of the arm's total budget, so there is almost nothing left to save. After cleaning, the direct-safe set and the planning-cheap set coincide

doi:10.6342/NTU202601985

at this scale, so routing at 20B is a per-sample-quality and exposure-control lever, not a compute lever. Among the arms whose rule the cleaned evidence supports, ROUTER-v2 solves its 21 tasks at the lowest cost; the planning-only $10/10$ arm buys its two extra tasks for 37.6 M further tokens, an edge Section 5.6 shows is sampling density rather than repair budget; and the brute-force HYBRID deployment stays dominated (20 tasks at 423 hours, though that comparison is directional, since it predates direct-flow repair and uses $10/10$ budgets).

Scale, and where a compute win could return. The comparison repeats at gpt-oss-120B, where both arms postdate the S-box fix. The v1 cascade solves 25 tasks at pass@1 32.7 % (cleaned; vs. 21 at 20B), and FUNC-10/10-120B, the planning-only $10/10$ configuration on the larger model (Table 5.5), solves 24 at pass@1 33.2 %, so scale helps genuinely on both sides. The two task sets are not nested. The cascade's two extra tasks are the integration-heavy `e203_cpu_top` and `e203_exu`, reached at budgets $6/4$; the planning arm's extra task is `e203_exu_decode`, which only its functional repair loop converts (Section 5.5). Because the arms differ in repair budgets, and the 120B serving deployment's wall-clock is not comparable to the 20B arms', the two 120B runs are compared on coverage and rates only, not as points on Figure 5.6's frontier. The re-derived rule's 120B profile widens the gates (ports $\leq$ 30, specification $\leq$ 12k characters, state $\leq$ 140 bits; 16 tasks route direct). Re-scoring the cleaned cascade samples under its decisions yields 26 of 60, with no solvable task routed away from its solving flow; the gain over those samples' planning flow is `e203_exu_oitf`, which the direct flow converts at 2–3 in 10. That task is no longer direct-exclusive, though. FUNC-10/10-120B reaches it through the planning flow at the deeper budgets. If routing is to recover a genuine compute win, the carve-out must find planning-expensive tasks the direct flow genuinely solves, a set

doi:10.6342/NTU202601985

that is empty at 20B and, once deep repair budgets are allowed, close to empty at 120B as well. Recomputing the oracle labels from cleaned evidence and a multi-seed replication are the outstanding controls (Section 6.3).

5.8 Failure Categorization: Three Tiers of Difficulty

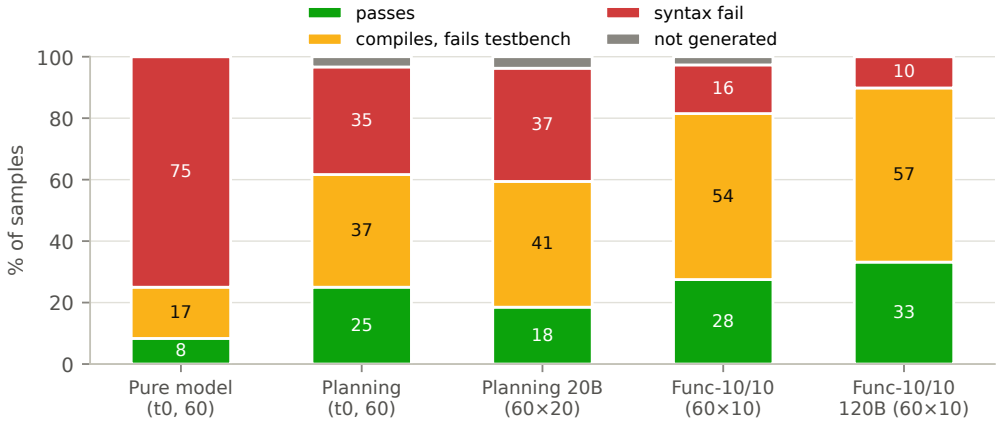


Figure 5.8: Per-sample outcomes. As grounding, repair budget, and model scale improve, the red syntax band shrinks — and the amber compiles-but-fails-testbench band absorbs it. The functional share of compiling samples barely moves. The two FUNC-10/10 arms are scored on the repaired S-box testbenches, the other three arms before the June-25 repair (Section 5.9).

Figure 5.8 decomposes every sample into four outcomes. The picture is consistent across runs. The largest failure bucket is not syntax but compiles-and-fails-the-testbench — right ports, right structure, wrong logic. Among compiling samples, that share is 59.5 % in PIPE-T0, 68.9 % in CACHE-OFF, 66.3 % in FUNC-10/10, and 63.1 % in FUNC-10/10-120B. Repair budget moves syntax, not semantics. FUNC-10/10 compiles 81.5 % of its samples against CACHE-OFF's 59.4 %, yet the functional share of its compiling samples barely shifts. Scale behaves the same way. The 6× larger PIPE-120B (not shown in the figure) is a strictly better Verilog syntactician — syntax 66.9 % → 76.3 % over its matched 20B arm (CACHE-RUN), first-pass compiles up, repair turns down — but it is not a better logician; its functional share stays at 67.9 %. The rightmost bar plays both levers at once and

doi:10.6342/NTU202601985

lands in the same place. FUNC-10/10-120B pairs the larger model with the deep budgets, and its non-functional failure modes all but disappear — 89.8 % of its samples compile, the syntax band is down to 10 %, and, alone among the planning arms, no sample fails to generate at all. Yet 63.1 % of what compiles still fails the testbench, so the amber band grows to 57 % of all samples, the largest in the figure. Neither budget nor scale fixes semantics, separately or combined; this is the strongest evidence that the functional ceiling is structural rather than a capability artifact of the 20B model.

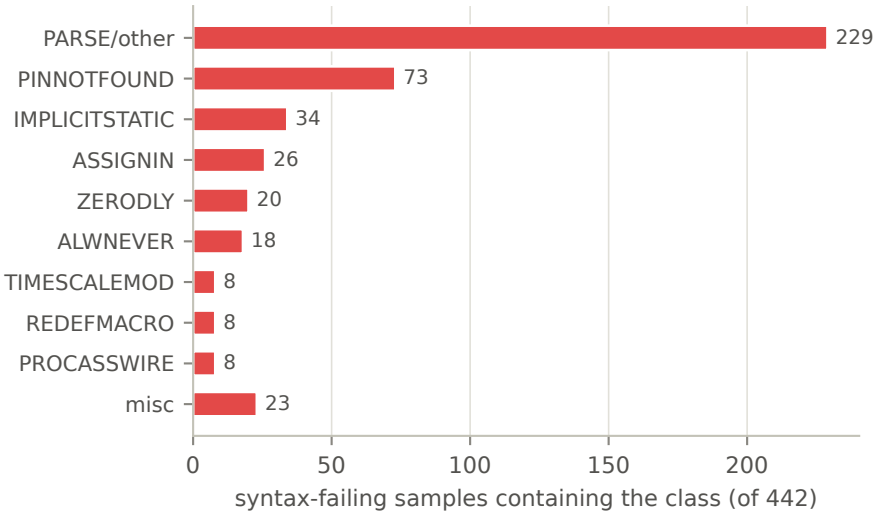


Figure 5.9: Verilator diagnostic classes among the 442 syntax-failing samples of CACHE-OFF (a sample can carry several classes).

Reading failures per module (Figures 5.9 and 5.10) resolves the aggregate into three stable tiers:

1. **Solved: leaf, RAM, and glue modules.** `aes_key_expand_128`, the CRC and RAM blocks, clock gating, register files — small, well-specified, often pure reuse. The pipeline has essentially retired this tier.

2. **Logic-limited: datapath and control modules.** Most AES round logic and the E203 ALU/decode/commit/dispatch cluster compile with perfect syntax and fail

doi:10.6342/NTU202601985

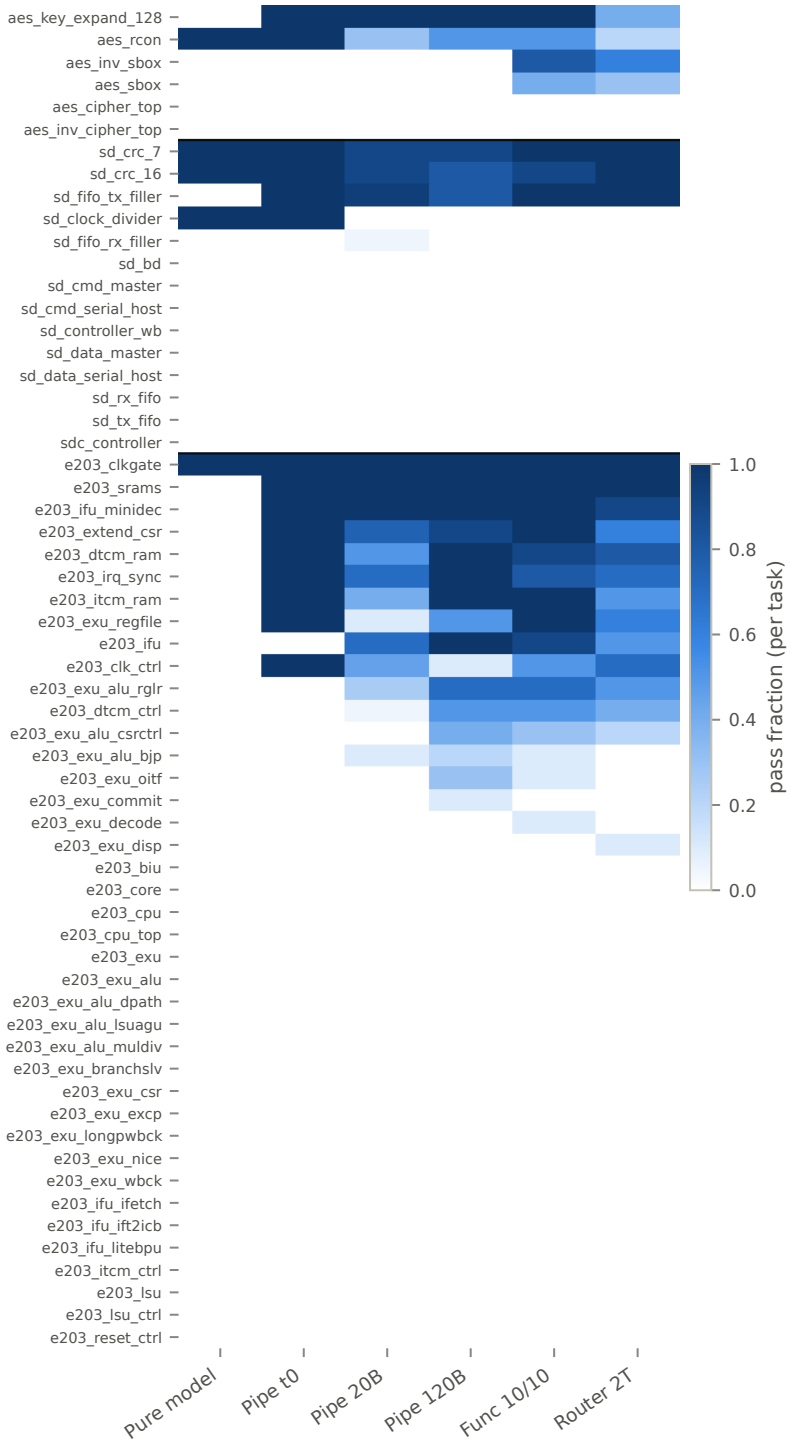


Figure 5.10: Pass fraction per task (rows, grouped AES/SDC/E203) across six runs (columns). The three difficulty tiers are visible as texture: saturated rows (leaf/glue modules), intermittent rows (logic-limited datapath modules), and blank rows (giant integrators). The two S-box rows are not comparable across columns, because the runs straddle the June-25 testbench repair (Section 5.9).

doi:10.6342/NTU202601985

the testbench — some at 0 passes despite dozens of structurally distinct compiling candidates. The syntax-only repair loop is blind here by construction.

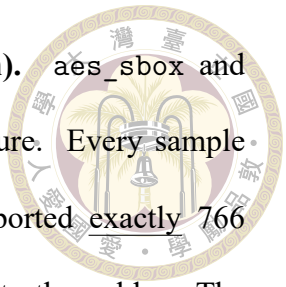
3. **Scale-limited: giant integrators.** `e203_core/cpu/exu/lsu`, `sdc_controller`: hundreds of ports, dozens of submodules. Failures are interface-completeness (`PINNOTFOUND` concentrates exactly here), context overflow (the not-generated bucket: timeouts, empty returns, and HTTP 400s on the biggest specs), and integration logic. Notably, plan-level interface hallucination reappears only on this tier. The `e203_exu` plan promotes 25 internal signals to phantom top ports while dropping 10 real ones — the same interface-too-large-to-hold phenomenon at a different stage.

Each tier wants a different intervention — nothing (tier 1), functional feedback (tier 2), decomposition and contract enforcement (tier 3). That is why aggregate pass rate alone is a blunt instrument for steering this system.

5.9 When the Score Is Wrong: Benchmark and Harness Artifacts

Treating every 0 % module as model failure would misdirect the next round of engineering, so the persistent zeros were audited individually with a golden-model methodology. Run the benchmark's own reference implementation through the full scoring path; if the golden fails (or a byte-equivalent candidate fails), the task's zeros are artifacts, not model errors. The same skepticism was then turned around and applied to the persistent passes. Four audits produced four different verdicts.

doi:10.6342/NTU202601985

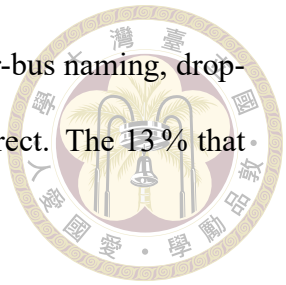


The AES S-box race (confirmed artifact, since fixed upstream). `aes_sbox` and `aes_inv_sbox` scored 0 across all runs with one suspicious signature. Every sample — combinational, ROM-array, or registered variants alike — reported exactly 766 mismatches. Diffing showed generated lookup tables byte-identical to the golden. The cause is a testbench race. Stimulus changes the input and samples the output in the same simulation timestep, so any correct-but-differently-scheduled implementation (e.g. a continuous `assign` where the reference uses blocking `always @(a)`) settles one delta cycle late at each of the $\sim$768 input transitions. The trigger was isolated by incremental flag bisection to Verilator's `--coverage-line` instrumentation; the identical design passes without it. The recorded mismatch count decomposes exactly as $766 + 3 \times (\text{wrong table bytes})$, so the artifact also corrupts mismatch counts as a repair signal — a one-byte-wrong design (0.4 % wrong) reports $\sim$50 % mismatches. A registered-output module using the same testbench template (`aes_key_expand_128`) passes 20/20, completing the case that this is a combinational-settling race. In CACHE-OFF alone, 21 byte-perfect samples scored zero, and 2 of 60 tasks were unsolvable purely by artifact. The finding was reported and the benchmark's testbench was repaired upstream on June 25, 2026 (settle before compare); pre-/post-fix runs are never compared on these two tasks.

The SDC controller (audited, genuine). `sdc_controller` — 0 functional passes in 220 records — looked like a candidate for the same treatment, and it is the reason the golden methodology matters. Its golden implementation passes its testbench perfectly (0 mismatches in 23,932 checks) under the full scoring flags, because all its outputs are registered and immune to the settling race. The zeros are honest. 87 % of SDC samples die on syntax through interface hallucination introduced at the generation stage (instantiating

doi:10.6342/NTU202601985

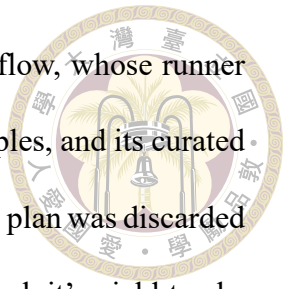
provided submodules with invented pins, inventing a different master-bus naming, dropping the `sd_defines` macros), even when the plan's interface is correct. The 13 % that compile are grossly wrong (∼95 % mismatch rates), not near misses.



The repair gate stricter than the scorer (pipeline-side finding). Auditing the 97 samples of FUNC-10/10 that exhausted all ten syntax repairs found that 42 of them (43 %) fail on a fatal warning located in a benchmark harness file (`stimulus_gen/ref`) — code the model cannot edit. The internal repair gate (`verilator --lint-only`) treats three warning classes as fatal that the actual scorer's build does not even emit, so on four tasks every sample burned its entire repair budget "fixing" correct code and never reached the functional phase. The golden test was run again. Three of the four goldens pass the real scoring build (pure pipeline artifact; the fix is to mirror the scorer's waivers in the gate), while the fourth (`e203_reset_ctrl`) fails even as golden — a genuine benchmark defect of the AES class. The remaining 53 % of repair-exhausted samples fail on real errors in generated code, concentrated on the giant integrators of tier 3.

The reference-hack leak (audited, our-side; all runs re-scored). The fourth audit inverted the question. Instead of asking whether zeros were honest, it asked whether the passes were. The direct flow's prompt lists the verification directory's provided helper modules so the model will not redeclare them. That list included `ref_<task>`, the benchmark's own golden reference model. The models found the exploit. A top module that merely instantiates `ref_<task>` compiles (the scoring Makefile globs the whole verification directory) and matches the reference by construction. Detection is mechanical — a generated sample is a ref-hack if any line instantiates a `ref_*` module — and unambiguous. Every flagged sample instantiates exactly its own task's reference, and all but one

doi:10.6342/NTU202601985



were generated from a prompt that named the module; the planning flow, whose runner filters `ref_*` names from its prompts, produced no hack in 7,760 samples, and its curated vocabulary's single escape — a probe-routed sample generated after its plan was discarded — reproduced the name from prior knowledge (Section 5.2). The exploit's yield tracks the machinery around it. In HYBRID, whose direct flow had no repair loop, only 48 of 186 attempted hacks compiled; in ROUTER-V1, with repair, 54 of 56 passed; at 120B, all 51 attempts passed. The impact was concentrated exactly where it was most misleading. In all, 13 of ROUTER-V1's 34 raw solved tasks and 10 of HYBRID's 30 were hack-only, manufacturing the apparent direct-flow advantage on FSM and algorithm tasks that the routing analysis had initially credited (Section 5.7). Every number in this thesis excludes hack samples (counted as syntax and functional failures). The fix on the generation side is one line — filter `ref_*` from the direct prompt's provided-module list, as the planning-flow runner already does — and hack screening is now part of our scoring path.

The methodological point generalizes beyond this benchmark. In a harness this strict, persistent zeros — and flattering passes — deserve a golden-model audit before they are allowed to steer engineering effort. Of our four audits, one cleared the models (and improved the benchmark upstream), one confirmed the difficulty as genuine, one located a configuration to align in our own gate, and one caught our own prompt handing the models a shortcut and corrected a headline ablation before it could mislead. Each verdict redirected the engineering correctly.

doi:10.6342/NTU202601985

Chapter 6 Discussion, Limitations, and Conclusion

6.1 Discussion

What actually moved the needle. Ranked by measured effect, the levers were as follows. (1) environment-faithful verification and repair — compiling against the real file set, injecting the testbench port contract, and condensing giant specs took the pipeline from low-single-digit to ∼18–25 % pass, the single largest step in the project's history; (2) model scale — 6× parameters bought +9 pp syntax and +4 pp pass@1 with the system held fixed; (3) routing — matching task character to flow shape held coverage at 21 of 60, level with the fitted rule it replaces, with better per-sample pass and syntax rates and no exposure to the benchmark's leaked reference models; the original rule's apparent compute cut did not survive the benchmark-gaming audit (Section 5.7); (4) repair caching — +5–7 pp syntax, density only. Two levers were smaller on the headline metric but valuable elsewhere — grounding (+0.6 pp, while buying auditability and eliminating a failure class) and the functional repair loop (+7 net passes, best run at a compact budget). The general lesson is plain. In agentic code generation for strict environments, fidelity between the inner loop and the scorer is worth more than additional intelligence in any

doi:10.6342/NTU202601985

single stage.

Grounding as a harness property. The planner's tool suite gives it evidence on demand — catalog search, candidate inspection, interface checks — but the design never lets plan correctness depend on any particular trajectory through those tools. The catalog vocabulary is also injected into the prompt, the output schema is constrained to that vocabulary, and every emitted reuse decision is checked against the catalog afterwards. The result is that plans are correct by construction regardless of how the model arrived at them, and the grounding audit (Section 5.2) can verify it decision by decision. This pattern — make correctness a property of the harness, layered over rather than delegated to the model's tool-use behavior — should transfer well beyond RTL.

What routing buys here. The routing literature mostly buys quality with compute along one axis (small model vs. large model). Our setting is different in kind. The expensive flow can do everything the cheap flow can, but the two differ by an order of magnitude in cost per sample, and the scaffold that makes the expensive flow strong on integration-heavy tasks is measurably counterproductive on self-contained ones (Section 3.5.1). The corrected result shrinks the payoff, not the direction. After the audit, the set the cheap flow genuinely handles is narrow and also happens to be planning-cheap, so at 20B routing buys per-sample quality and exposure control rather than compute; the compute lever returns only where the cheap-safe set widens into planning-expensive territory, and even the first candidate at 120B turns out to be reachable by the deep-budget planning flow as well (Section 5.7). The setting's shape will recur wherever an elaborate agentic scaffold coexists with a cheap direct path, because the routing signal (does this task's difficulty live in its interfaces or in its logic?) can be read off the specification itself. Our audit adds

doi:10.6342/NTU202601985

a more general lesson. When a cheap flow suddenly looks more able than the scaffold, rather than merely cheaper, audit that claim before believing it — and before fitting any rule to it (Section 5.9).

The functional ceiling. The binding constraint at the end of this work is clear. ∼60–69 % of everything that compiles fails its testbench, a share that model scale, caching, and ten rounds of oracle-fed repair barely move. Verilator diagnostics give the repair loop a precise, local, actionable signal for syntax; a mismatch count gives almost nothing — and Section 5.9 showed it can even be actively misleading. Progress on this tier will require a fundamentally better feedback channel (waveform-level localization, per-submodule benches, assertion synthesis), not more attempts on the current one.

6.2 Threats to Validity

This section collects every caveat that qualifies the numbers in this thesis; each was also flagged inline where it applies.

1. **Single-seed runs.** Temperature-0 arms are one sample per task; each sampled run is one seed. Cross-run deltas under ∼2–3 pp are compatible with sampling noise. The claims we rest weight on (headline 3.0×; the flat pass@20; ROUTER-v2's per-sample pass@1 and syntax margins over the v1 rule, measured on 600 samples each) are far outside that band — the routing arms' one-task set differences are inside it and are treated as noise.

2. **Cross-run configuration drift.** Only four comparisons are single-variable: the cache trio, the temp-0 head-to-head, ROUTER-v1→ROUTER-v2 (decision rule only),

doi:10.6342/NTU202601985

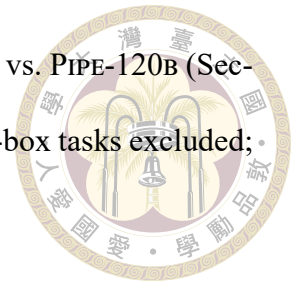
and FUNC-10/10→FUNC-10/10-120B (model scale only, though the two ran on different serving deployments, so only quality metrics — not wall-clock — are compared). Everything else — notably the router-vs-HYBRID comparison (repair budgets and direct-flow repair differ) and the FUNC-2/4 →FUNC-10/10 pair (a scoring waiver changed) — is labeled directional where it appears.

3. **Cost accounting.** Token totals from resumed runs undercount, because resumed samples log zero tokens; within the cache trio the only clean full-run token cost is CACHE-OFF's 46.3 M (the FUNC arms are uninterrupted full runs, so their totals are clean). Direct-flow repair calls are not instrumented, so direct-flow token totals — including ROUTER-V1's 28.3 M and ROUTER-V2's 52.2 M — are lower bounds. No headline rests on them. Section 5.7 already sets aside the v1 arm's per-solved-task advantage as fitted to pre-audit evidence, and full instrumentation would only raise ROUTER-V2's total (its 110 direct samples are the only uninstrumented part), leaving the ≈all-planning-cost conclusion unchanged. Wall-clock is never compared across days (shared server, varying load).

4. **Scoring strictness.** The harness fails on warnings, so our syntax rates are conservative relative to error-only benchmarks — and Section 5.9 showed our own internal gate was at times stricter than the scorer, deflating four tasks.

5. **Benchmark fix boundary.** The AES S-box testbenches were repaired upstream on June 25, 2026, after our report, and the run inventory straddles that date. DIRECT-T0, PIPE-T0, the cache trio, PIPE-120B, and FUNC-2/4 were scored on the broken benches (their S-box failures carry the race's 766-mismatch signature), while FUNC-10/10, FUNC-10/10-120B, HYBRID, and every router arm postdate the fix. Comparisons whose arms sit on the same side are unaffected. Two comparisons span the bound-

doi:10.6342/NTU202601985



ary — FUNC-2/4 → FUNC-10/10 (Section 5.5) and FUNC-10/10 vs. PIPE-120B (Section 5.3) — and both are therefore also reported with the two S-box tasks excluded; no other conclusion rests on those two tasks.

6. **Ref-hack exclusion.** All rates exclude samples that instantiate a leaked `ref_*` reference model (Section 5.9). Detection is a syntactic screen over generated RTL; it flagged only unambiguous self-hacks and cannot, by construction, catch subtler uses of the leaked name — though the reference source was never visible to the models, bounding what a subtler exploit could copy. The archived pre-fix runs retain the leak in the direct-flow prompt; ROUTER-V2 already runs with the filtered prompt (and produced zero hack samples), and re-running the remaining direct-flow arms with it is future work (Section 6.3).

7. **Oracle feedback.** The functional repair loop consumes the scoring testbench's mismatch report. Its passes mean "specification plus oracle feedback" and are quarantined in Section 5.5; no headline number includes them. Relatedly, the internal lint compiles the testbench, so Verilator diagnostics can quote testbench lines into repair prompts; a flag exists to disable this for leak-free claims, at a syntax-rate cost we did not pay here.

8. **Benchmark scope.** All results are on one benchmark's 60 module-level tasks and three IP projects, with open-weight models of one family for all controlled comparisons. The family-level heterogeneity we observe (AES vs. E203) is itself evidence that results will vary across codebases.

doi:10.6342/NTU202601985

6.3 Future Work

Five directions follow directly from the measured constraints. First, functional feedback without oracle leakage. Synthesize self-checking benches or assertions from the specification, so the functional repair loop gets a mismatch signal the scorer never sees — and gate it off pure-combinational modules, where Section 5.9 showed the signal can be corrupted. Second, combine the pipeline with a testbench-generation pipeline to unlock fine-grained module-decomposition IP reuse. Today the plan's decomposition stops at module granularity because only the top module has an oracle. A companion pipeline that generates a bench per planned submodule would let the system decompose more aggressively, verify each decomposed piece independently, and reuse IP at the sub-block level, turning the planner's decomposition from a prompt-structuring device into a verified construction process. Third, interface-contract enforcement as a hard gate. The testbench DUT instantiation is already injected as text; promoting it to a mechanical post-generation port-diff check would remove the `PINNOTFOUND` class that dominates giant-module syntax failures, and the internal repair gate should mirror the scorer's warning waivers exactly. Fourth, selective decomposition for the giants. RealBench's system-level tasks decompose into submodules that are themselves benched module tasks, so a tree-structured generator could verify each node against a real testbench and compose upward. This is decomposition with per-node oracles, reserved for the scale-limited tier where it pays, and the natural first customer of the testbench-generation direction above. Fifth, hardening the router. Recompute the oracle label table from cleaned evidence so that confusion matrices are citable again; search for planning-expensive tasks the direct flow genuinely solves — the only place a routing compute win can come from, a set that stays close to empty at

doi:10.6342/NTU202601985

120B once deep repair budgets are allowed (Section 5.7); extend the `ref_*`-filtered direct prompt — already validated in the ROUTER-v2 run, which produced zero hack samples — to re-runs of the archived direct-flow arms as the definitive control for the ref-hack exclusion of Section 5.9; and replicate across seeds to make the routing result statistically settled.

6.4 Conclusion

This thesis asked how far careful system engineering — planning, grounding, verification, repair, and routing around fixed open-weight models — can push LLM-based RTL generation on a realistic, reuse-dominated benchmark. The answer is a long way, and measurably so. A two-stage pipeline that plans IP reuse against a grounded per-task catalog and generates under environment-faithful verification reaches $2.5\times$ the syntax rate and $3.0\times$ the pass rate of the same model prompted directly, with the entire gain concentrated where reuse exists to be exploited. A per-task router whose decision rule was re-derived on the audited solution set keeps the fitted rule's coverage at 21 of 60 tasks with higher per-sample pass and syntax rates and no exposure to the benchmark's leaked reference models.

The thesis's second kind of contribution is measurement discipline over the same system. Grounding made every plan correct and auditable. Caching concentrated passing density inside solvable tasks. The repair budgets were calibrated to where the rescues actually happen, recovering the large majority of the functional phase's compute. A golden-model audit of persistent zeros cleared the models on two tasks (and improved the benchmark upstream), caught our own repair gate being stricter than the scorer on three

doi:10.6342/NTU202601985

more, exposed a further benchmark defect on a fourth, and confirmed the remaining zeros as genuine difficulty. Turned on our own passes, the same audit caught a prompt-side leak that had inflated the routing ablation, corrected it before it could mislead, and led to the routing rule being re-derived on the corrected evidence. These findings redirected the engineering each time, and one was adopted upstream by the benchmark itself. The frontier, at the end, is not syntax, not plans, and not scale. It is functional correctness, and the next system to move it will be the one that finds its models a better feedback signal than a mismatch count.

doi:10.6342/NTU202601985

References

[1] P. Aggarwal, A. Madaan, A. Anand, S. P. Potharaju, S. Mishra, P. Zhou, A. Gupta, D. Rajagopal, K. Kappaganthu, Y. Yang, S. Upadhyay, M. Faruqui, and Mausam. AutoMix: Automatically mixing language models. In Advances in Neural Information Processing Systems 37 (NeurIPS), 2024. arXiv:2310.12963.

[2] A. Allam and M. Shalan. RTL-Repo: A benchmark for evaluating LLMs on large-scale RTL design projects. In 2024 IEEE LLM Aided Design Workshop (LAD), 2024. arXiv:2405.17378.

[3] J. Blocklove, S. Garg, R. Karri, and H. Pearce. Chip-chat: Challenges and opportunities in conversational hardware design. In 2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD), 2023. arXiv:2305.13243.

[4] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. Transactions on Machine Learning Research, 2024. First appeared as arXiv:2305.05176 (2023).

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, et al. Evaluating large language models trained on code, 2021.

doi:10.6342/NTU202601985

[6] M.-C. Chen, Y.-H. Kao, P.-H. Huang, S.-C. Ho, H.-Y. Tsou, I.-T. Wu, E.-M. Huang, Y.-K. Hung, W.-P. Hsin, C. Liang, C.-H. Tu, S.-H. Hung, and H. T. Kung. SiliconMind-V1: Multi-agent distillation and debug-reasoning workflows for Verilog code generation, 2026.

[7] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, and A. H. Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In The Twelfth International Conference on Learning Representations (ICLR), 2024. arXiv:2404.14618.

[8] C.-T. Ho, H. Ren, and B. Khailany. VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 39, pages 300–307, 2025. arXiv:2408.08927.

[9] IEEE. IEEE standard for SystemVerilog—unified hardware design, specification, and verification language. IEEE Std 1800-2023, 2023.

[10] P. Jin, D. Huang, C. Li, S. Cheng, Y. Zhao, X. Zheng, J. Zhu, S. Xing, B. Dou, R. Zhang, Z. Du, Q. Guo, and X. Hu. RealBench: Benchmarking Verilog generation models with real-world IP designs, 2025.

[11] M. Keating and P. Bricaud. Reuse Methodology Manual for System-on-a-Chip Designs. Kluwer Academic Publishers, Boston, MA, 3rd edition, 2002.

[12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2023.

doi:10.6342/NTU202601985

[13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems 33 (NeurIPS), 2020. arXiv:2005.11401.

[14] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu, et al. ChipNeMo: Domain-adapted LLMs for chip design, 2023.

[15] M. Liu, N. Pinckney, B. Khailany, and H. Ren. VerilogEval: Evaluating large language models for Verilog code generation. In 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD), 2023. arXiv:2309.07544.

[16] M. Liu, Y.-D. Tsai, W. Zhou, and H. Ren. CraftRTL: High-quality synthetic data generation for Verilog code models with correct-by-construction non-textual representations and targeted code repair. In The Thirteenth International Conference on Learning Representations (ICLR), 2025. arXiv:2409.12993.

[17] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie. RTLCoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution. In 2024 IEEE LLM Aided Design Workshop (LAD), 2024. arXiv:2312.08617.

[18] S. Lu, N. Duan, H. Han, D. Guo, S. won Hwang, and A. Svyatkovskiy. ReACC: A retrieval-augmented code completion framework. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 6227–6240. Association for Computational Linguistics, 2022.

[19] Y. Lu, S. Liu, Q. Zhang, and Z. Xie. RTLLM: An open-source benchmark for design

doi:10.6342/NTU202601985

RTL generation with large language model. In Proceedings of the 29th Asia and South Pacific Design Automation Conference (ASP-DAC), 2024.

[20] Nuclei System Technology. Hummingbirdv2 E203 ultra-low power RISC-V processor core. https://github.com/riscv-mcu/e203_hbirdv2, 2026. Open-source project. Accessed July 2026.

[21] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is self-repair a silver bullet for code generation? In The Twelfth International Conference on Learning Representations (ICLR), 2024. arXiv:2306.09896.

[22] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs from preference data. In The Thirteenth International Conference on Learning Representations (ICLR), 2025. arXiv:2406.18665.

[23] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025.

[24] OpenCores. OpenCores: Open-source IP cores repository. https://opencores.org, 2026. Accessed July 2026.

[25] H. Pearce, B. Tan, and R. Karri. DAVE: Deriving automatically Verilog from English. In Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD (MLCAD), pages 27–32, 2020.

[26] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany. Revisiting VerilogEval: A year of improvements in large-language models for hardware code generation. ACM Transactions on Design Automation of Electronic Systems, 30(6):91:1–91:20, 2025. arXiv:2408.11053.

doi:10.6342/NTU202601985

[27] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems 36 (NeurIPS), 2023. arXiv:2303.11366.

[28] W. Snyder. Verilator: Open-source SystemVerilog simulator and lint system. https://verilator.org, 2026. Accessed July 2026.

[29] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg. VeriGen: A large language model for Verilog code generation. ACM Transactions on Design Automation of Electronic Systems, 29(3):46:1–46:31, 2024.

[30] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri. AutoChip: Automating HDL generation using LLM feedback, 2023.

[31] Y.-D. Tsai, M. Liu, and H. Ren. RTLFixer: Automatically fixing RTL syntax errors with large language model. In Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC), 2024.

[32] C. Wolf and J. Glaser. Yosys – A free Verilog synthesis suite. In Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip), 2013.

[33] K. Xu, J. Sun, Y. Hu, X. Fang, W. Shan, X. Wang, and Z. Jiang. MEIC: Re-thinking RTL debug automation using LLMs. In Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2024. arXiv:2405.06840.

[34] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, et al. Qwen3 technical report, 2025. Qwen Team, Alibaba Group.

[35] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press.

doi:10.6342/NTU202601985

SWE-agent: Agent-computer interfaces enable automated software engineering. In Advances in Neural Information Processing Systems 37 (NeurIPS), 2024.

[36] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In The Eleventh International Conference on Learning Representations (ICLR), 2023. arXiv:2210.03629.

[37] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 2471–2484. Association for Computational Linguistics, 2023.

[38] Y. Zhao, D. Huang, C. Li, P. Jin, Z. Nan, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang, X. Zhang, Z. Du, Q. Guo, X. Hu, and Y. Chen. CodeV: Empowering LLMs for Verilog generation through multi-level summarization, 2024.

doi:10.6342/NTU202601985

Appendix A — Prompt Templates

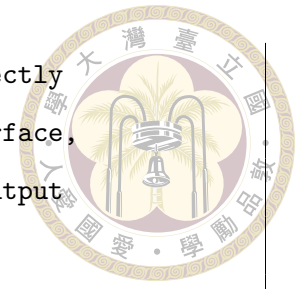
This appendix reproduces the prompt text behind every LLM call in the system, so that the mechanisms described in Chapter 3 can be audited at the level at which they actually operate. Text is excerpted verbatim from the repository (`agentic_ip_reuse/.../prompts.py`, `ip_reuse_legacy/prompts.py`, `rag_rtl/routing.py`, and the runner scripts). Ellipses mark omissions, values in curly braces are placeholders the harness fills at run time, and long lines are re-wrapped for the page.

A.1 Planner System Prompt

The planner's system prompt walks the model through the eight planning stages, declares the tool suite of Section 3.2, and — decisive in practice — mandates the closed catalog vocabulary. The grounding pass described in that section enforces the same contract on the returned plan after the fact, so the closed vocabulary holds regardless of the trajectory the planning agent takes through its tools.

```
You are an agentic IC design and IP-reuse planning assistant.
Complete these stages in order:
1. Decide all system-level requirements: functionality,
   performance target, I/O interface, and PPA constraints.
2. Decompose the system into modules. Always check if the spce
```


doi:10.6342/NTU202601985



```
   supply the sub-module list or not, if does supply, directly
   use it and forward to step 3, else consider Input Interface,
   Buffer/FIFO, Processing Core, Memory Controller, and Output
   Interface.
   ...
3. Search for reusable IP. Evaluate every candidate against
   function match, interface compatibility, configurability,
   verification status, license, synthesis support, and
   documentation quality.
4. Understand selected IP interfaces and behavior before
   integration.
5. Configure or parameterize selected IPs (data width, buffer
   depth, modes, clock assumptions).
6. For modules requiring new RTL: call generate_rtl_module with
   the complete synthesizable SystemVerilog code. Then call
   validate_verilog on the generated file. If validation returns
   errors, fix them and call generate_rtl_module again.
7. For each pair of connected modules, call
   check_port_compatibility to confirm direction and bit-width
   match before writing the integration plan.
8. Plan simulation, synthesis, and debugging.

Available tools:
- search_reuse_ip: search the local reusable-IP catalog.
- inspect_reuse_ip: return behavior, interfaces, parameters,
  limits, and integration notes for one IP.
- evaluate_ip_candidate: score a candidate against module
  requirements.
- write_artifact: write Markdown or JSON artifacts into the
  output directory.
- generate_rtl_module: write a complete synthesizable
  SystemVerilog module to a .sv file. Provide the full RTL in
```


doi:10.6342/NTU202601985

```
  the verilog_code parameter.
- validate_verilog: lint a generated .sv file (uses verilator or
  iverilog). Returns errors and warnings. Self-correct and
  regenerate if errors are found.
- check_port_compatibility: parse port declarations from two .sv
  files and verify direction and width compatibility.

CATALOG GROUNDING (mandatory): The reusable-IP catalog for this
task is listed verbatim in the user message under "Reusable IP
catalog". Treat it as the only source of truth for which IPs
exist.
- Every reuse_decisions entry MUST set "selected_ip" to an
  "ip_id" that appears EXACTLY in that list.
- Never invent, rename, or add suffixes to an IP name (e.g. do
  not turn "e203_exu" into "e203_exu_core").
- If no catalog IP fits a module, set "new_rtl_required": true
  and "selected_ip": null for that module instead of guessing a
  name.
- Produce one reuse_decisions entry for every module that has a
  candidate IP in the catalog.

If no IP satisfies reuse criteria for a module, mark it as
new_rtl_required, explain which criteria failed, then generate
the RTL with generate_rtl_module.
Return final output as one JSON object only, with keys:
requirements, modules, reuse_decisions, integration_plan,
verification_plan, debug_plan, unresolved_assumptions.
Every reply must either call a tool or contain that final JSON
object. Never reply with working notes, intentions, or partial
reasoning alone.
```

Listing A.1: Planner system prompt.

doi:10.6342/NTU202601985

A.2 Planner User Prompt and Catalog Digest

The user message carries the specification (the planner view when condensation ran), the injected catalog digest — one line per provided module with its `ip_id`, one-sentence summary, and leading port names — and a closing restatement of the closed-vocabulary rule.

```
Target HDL: {target_hdl}

Known constraints:
- {constraint lines, or "Follow the user request exactly."}

Known interfaces:
- {interface lines, or "Discover from the user request."}

PPA targets:
- {target lines, or "Infer and list assumptions."}

User request:
{specification; the planner view when condensation ran}

Reusable IP catalog (authoritative - select reuse IPs ONLY from
these ip_id values):
- {ip_id}: {one-line summary} | ports: {first 12 port names}
- ...
- (+{N} more catalog entries omitted)

Produce a complete IP-reuse design plan. Every
reuse_decisions.selected_ip must be one of the ip_id values
listed in the catalog above; do not invent names. Use the
catalog tools when they can improve reuse decisions.
```


doi:10.6342/NTU202601985

Listing A.2: Planner user prompt with the injected catalog digest.

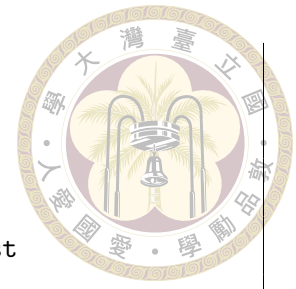
A.3 Planner View vs. Generation View of a Condensed Specification

Large specifications are condensed once into a chunk-and-merge manifest (Section 3.2) and then rendered into two different textual views, one per consumer. Both views share the same skeleton, in this order: top module name; system summary; the top module's public interface; its submodule instances and connections; shared clocks, resets, parameters, constraints, and assumptions; authoritative interface excerpts carried verbatim from the raw specification (with an instruction to trust them over any summarized interface); every module's interface listed exactly once; and finally per-module behavioral requirements. The two views differ in exactly the dimensions their consumers need:

```
SHARED SKELETON (both views)
  TOP MODULE / SYSTEM SUMMARY
  TOP PUBLIC INTERFACE / SUBMODULE INSTANCES
  SHARED CLOCKS, RESETS, PARAMETERS, CONSTRAINTS
  AUTHORITATIVE INTERFACE EXCERPTS (verbatim, trusted)
  MODULE INTERFACES (each module exactly once)
  MODULE BEHAVIORAL REQUIREMENTS

PLANNER VIEW (route/reuse decisions)
  + per-module "reuse_query:" line -- a ready-made catalog
    search query for each module
  + behavioral requirements digested to the first 3 items
```


doi:10.6342/NTU202601985



```
   per module ("... plus N more requirement(s)")

GENERATION VIEW (writing the RTL)
  + full behavioral requirements for every module that must
    be generated
  + modules named as provided IP are interface-only, tagged
    "[provided reusable IP - instantiate, do not regenerate]"
  + section order: to-generate modules first (in dependency
    order), provided IP last -- so head-truncation under a
    context budget keeps what the generator has to build
```

Listing A.3: Planner view vs. generation view: the rendered differences.

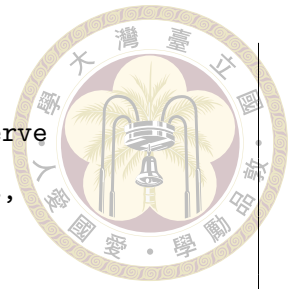
The planner view is thus a reuse-structure lens built from interfaces, reuse queries, and a short behavioral digest. The generation view is a construction lens, with complete requirements for new logic and interface-only contracts for everything provided. This division is also what makes the plan informative for routing (Section 3.5.3). Module characteristics that are ambiguous in the raw specification arrive in the plan already separated into delegation structure and own behavior.

A.4 Specification Condensation Prompts

The manifest behind both views is built by two LLM stages (Section 3.2). Each chunk of the raw specification is first reduced to one-layer implementation facts, and the partial extracts are then merged into one canonical manifest. Both prompts forbid generating RTL and forbid inventing facts, and both keep the decomposition to the top module's immediate children.

```
Extract one-layer implementation facts from chunk {k} of {n} of
```


doi:10.6342/NTU202601985



```
a large hardware specification.
Do not generate RTL and do not invent missing facts. Preserve
exact names, ports, widths, connections, protocols, clocks,
resets, parameters, conditional requirements, and module
dependencies. Use "unknown" for unspecified values.
The expected top module is {top_module}. Target HDL is
{target_hdl}.
Extract only immediate children of the top module; do not
flatten grandchildren into the same module list.

Specification chunk:
{chunk}

Return only JSON. Use the same manifest fields when present:
system_summary, clocks, resets, parameters, shared_constraints,
assumptions, unknowns, top_module, and modules. Omit fields that
this chunk does not describe.
```

Listing A.4: Chunk-partition prompt.

```
Merge partial large-hardware-specification extracts into one
canonical one-layer implementation manifest.
Do not generate RTL. Do not invent facts. Deduplicate compatible
facts and preserve all exact requirements.
Resolve conflicts only when one extract is clearly more
specific; otherwise record the conflict in unknowns.
The caller-required top module is {top_module} and must be used
exactly when provided.
Keep only the top module's immediate children in modules. Do not
flatten deeper descendants into this manifest.

Target HDL: {target_hdl}
Extra constraints:
```


doi:10.6342/NTU202601985

```
{constraint lines}

Partial extracts:
{partial manifests, JSON}

Return only JSON using this complete shape:
{ system_summary, clocks, resets, parameters,
  shared_constraints, assumptions, unknowns,
  top_module: { name, ports (name/direction/width/description),
                instances (module/instance_name/connections) },
  modules: [ { name, category, purpose, ports,
               behavioral_requirements, dependencies,
               reuse_query } ] }
```

Listing A.5: Manifest-merge prompt (the required JSON shape is abridged).

A validation pass runs after the merge; when it reports errors, a correction prompt returns the manifest and the error list and asks for a corrected manifest under the same no-invention rules.

A.5 RTL Generation Prompt and the Shared Context Block

The generator prompt (Section 3.3) has a fixed header and a context block. The header states the task, the reuse rules, and the output format; the context block carries the provided-module signatures, the compile-environment notes — including the testbench DUT instantiation as the binding port contract — and the specification; the adapted plan closes the prompt. The same context block reappears in both repair prompts (Sections A.6 and A.7), where diagnostic-driven slicing may replace the specification inside it (Sec-

doi:10.6342/NTU202601985

tion 3.4.3).

```
Generate integrated {target_hdl} RTL from this IP reuse plan.
Use selected reusable IP behavior where appropriate, configure
or adapt candidates as described, and create new RTL for modules
marked new.
Reused modules listed as provided source files must only be
instantiated, never re-declared in your output.
Return exactly one fenced {target_hdl} code block and no extra
prose.

Top module: {top_module}

{shared context block, Listing A.7}

IP reuse plan:
{adapted plan, JSON}
```

Listing A.6: RTL generation prompt, header and skeleton.

```
Provided reusable modules (each is supplied as a separate source
file in the compile environment; instantiate them using these
exact module names and port declarations; do NOT re-declare or
re-implement any of them):
{module -> port-signature map, JSON}

Compile environment notes:
- The verification testbench instantiates {top_module} exactly
  as shown below. Your module must declare every port connected
  here, with these exact names (a missing port is a fatal
  PINNOTFOUND elaboration error). Ports inside `ifdef blocks are
  required whenever the compile environment defines that macro:
  {testbench DUT instantiation, verbatim}
- The compile directory provides these defines/include files:
```


doi:10.6342/NTU202601985

```
  {file names}. Reference their macros with a backtick (e.g.
  `MACRO_NAME) and put the matching `include directive(s) at the
  top of your file if you use any of those macros.
- These dependency modules are NOT provided by the compile
  environment and their full implementation must be included in
  your output: {module names}. Copy the original source from the
  reuse plan candidates verbatim when available.
- All modules listed under provided reusable modules are
  compiled from their own source files; emitting a module with
  one of those names will cause a fatal duplicate-declaration
  error.

Original design specification (authoritative for the top module
name, exact port names, directions, widths, parameters, reset
polarity, and behavior - follow it even where the plan
disagrees):
{specification; the generation view when condensation ran}
```

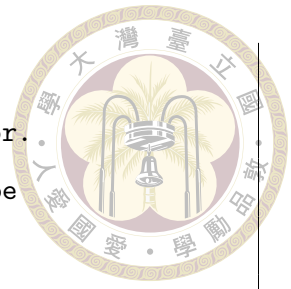
Listing A.7: The shared context block. The first three environment notes appear when the task provides the corresponding files; the specification is swapped for the diagnostic-driven slice in repair prompts when slicing is on.

A.6 Syntax Repair Prompt

When the internal lint fails, the failed tool reports are fed back verbatim together with the current candidate (Section 3.4.1). The guidance block appears only on a semantic repair-cache hit and is marked advisory; the diagnostics block carries each failed tool's name, verdict, and raw output, which is where the Verilator error lines the model must act on arrive.

```
Repair this integrated {target_hdl} RTL so it passes syntax and
```


doi:10.6342/NTU202601985



```
lint checks.
Keep the same IP reuse intent and public top-level behavior.
Reused modules listed as provided source files must only be
instantiated, never re-declared in your output.
Return exactly one fenced {target_hdl} code block and no extra
prose.

Top module: {top_module}

{shared context block, Listing A.7; the
 specification inside it is replaced by the interface-topic
 slice when slicing is on}

IP reuse plan:
{adapted plan, JSON}

Guidance from previously verified fixes of similar diagnostics
(advisory only; adapt the pattern, do not copy unrelated code or
module names):
{cached unified-diff excerpts, on a repair-cache hit}

Diagnostics:
{failed tool reports, JSON: tool, verdict, raw stdout/stderr}

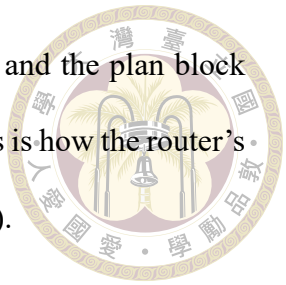
Current RTL:
```{target_hdl}
{current candidate}
```
```

Listing A.8: Syntax repair prompt.

When the repair loop runs on the direct flow (no plan), the reuse-intent line becomes

doi:10.6342/NTU202601985

"Keep the public top-level behavior described in the specification.", and the plan block and reuse rules are dropped; the rest of the skeleton is unchanged. This is how the router's direct arm shares the planning flow's repair machinery (Section 3.5.3).



A.7 Functional Repair Prompt

The functional repair turn (Section 3.4.2) re-frames the task as logic repair on a design whose interface is already correct. The simulator's mismatch report is inserted verbatim, and the behavior-topic slice replaces the specification when slicing is on. The same direct-flow variant as in Section A.6 applies.

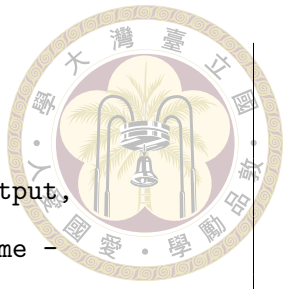
```
This integrated {target_hdl} RTL compiles cleanly but produces
outputs that do not match the reference testbench. Fix the
internal logic so the outputs match.
Do NOT change the module's port interface (names, directions,
widths) - it already matches the testbench, so changing it would
regress. Correct only the behavior/logic.
Keep the same IP reuse intent. Reused modules listed as provided
source files must only be instantiated, never re-declared in
your output.
Return exactly one fenced {target_hdl} code block and no extra
prose.

Top module: {top_module}

{shared context block, Listing A.7; the
 specification inside it is replaced by the behavior-topic
 slice when slicing is on}

IP reuse plan:
```


doi:10.6342/NTU202601985



```
{adapted plan, JSON}

Reference testbench mismatch report (each line names an output,
its mismatch count, and the first mismatched simulation time -
concentrate on the logic that drives those outputs around that
time):
{verbatim simulator report, e.g. "Output 'b' has 23 mismatches.
 First mismatch occurred at time 115."}

Current RTL:
```{target_hdl}
{current candidate}
```
```

Listing A.9: Functional repair prompt.

A.8 Direct-Flow Generation Prompt

The pure-model baseline and the router's direct arm (Sections 4.4 and 3.5.3) use one prompt, assembled by the direct runner. The provided-helper note names only genuine dependency modules; evaluation collateral — the `ref_*` golden model, testbench, and stimulus — is excluded from that vocabulary, so the prompt never names the files behind the reference-hack loophole audited in Section 5.7.

```
Generate the requested RealBench RTL implementation directly
from the problem statement.

Target HDL: {target_hdl}
Required public top module name: {top_module}

Constraints:
```


doi:10.6342/NTU202601985

```
{task constraint lines}
- Return only synthesizable RTL source code.
- Do not include a testbench, explanations, markdown fences, or
  analysis text.

The RealBench verification directory already provides these
helper modules. Do not redeclare them in your output unless the
problem explicitly requires a replacement: {module names}.

### RealBench Problem
{problem specification, clipped to the prompt budget}
```

Listing A.10: Direct-flow generation prompt.

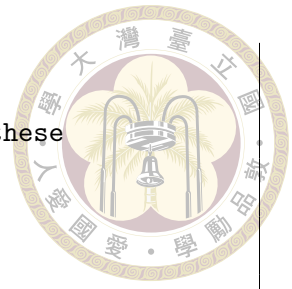
A.9 Router Feature-Extraction Prompt

Tier-0 of the cascade router (Section 3.5.3) is one LLM call. The specification is appended after this header, and the returned JSON carries the seven judgments plus a confidence value; the keyword ablation baseline extracts the same fields by pattern matching.

```
You are a routing feature extractor for an RTL code-generation
pipeline.
Read the ENTIRE Verilog module specification below and judge,
for THIS module only, whether the substantive logic must be
IMPLEMENTED in this module or is DELEGATED to instantiated
sub-modules / is trivially thin.

A module can SOUND like it computes (e.g. "preliminary
decoding") yet actually be a wrapper that instantiates another
module and passes signals through. Read the whole spec
(including Registers / Submodule sections) before deciding.
```


doi:10.6342/NTU202601985



```
Return ONLY a single JSON object, no prose, with exactly these
fields:
{
  "delegates_to_submodules": boolean, // mainly instantiates/
                  // encapsulates/integrates sub-modules
  "is_memory": boolean, // RAM / memory encapsulation
  "thin_control": boolean, // pure combinational control or
                  // field routing; "does not perform
                  // calculations"
  "has_fsm": boolean, // multi-state finite state
                  // machine / multi-cycle control
  "has_cdc": boolean, // clock-domain crossing / async
                  // FIFO / synchronizers
  "has_algorithm": boolean, // datapath algorithm: encryption
                  // rounds, CRC, full instruction decode,
                  // address arithmetic
  "est_state_bits": integer, // rough number of state/register
                  // bits THIS module itself must hold
                  // (0 for pure wrappers)
  "confidence": number // 0..1 confidence in the above
}

SPECIFICATION:
{full module specification}
```

Listing A.11: Tier-0 routing feature-extraction prompt.

doi:10.6342/NTU202601985

A.10 Specifications Routed as Uncertain

Tier-0 of the cascade router deferred exactly five of 60 tasks to the plan probe in the full-cascade run (Section 5.7). For each, the LLM feature extractor returned no class-A signal (delegation, memory encapsulation, thin control) and no class-B signal (FSM, CDC, datapath algorithm), at full confidence — the specifications genuinely commit to neither class. Their opening self-descriptions show the common character:

`sd_bd` "manages the buffer descriptors used for data transmission and reception between the system memory and the SD card … implements a circular buffer of descriptors" — a descriptor store with pointer bookkeeping, neither a wrapper nor a nameable algorithm.

`sd_data_serial_host` "the interface towards the physical SD card device Data port" — a serial engine mixing protocol timing, CRC handshake, and bus-facing buffering.

`e203_biu` "controls the ICB requests to the external memory system … serves as a crucial interface between the LSU, IFU, various peripheral interfaces, and the main memory" — pure interface arbitration between protocols.

`e203_exu_alu_lsuagu` "implements the AGU for load/store and AMO instructions … shares most of its datapath with the ALU" — address generation entangled with a shared datapath, neither self-contained nor delegating.

`e203_exu_wbck` "arbitrates between the write-back requests from the ALU and long-pipeline instructions" — an arbiter with modest own state, no submodules, and no algorithm by name.

doi:10.6342/NTU202601985

The shared shape — interface-arbitration modules that mediate between protocols — is a coherent third type of question sitting between the two classes. It is exactly the population the plan probe is designed to rule on, because once a plan exists, the presence or absence of delegation structure becomes explicit.

doi:10.6342/NTU202601985

doi:10.6342/NTU202601985

Appendix B — Per-Module Results and Reproduction

B.1 Per-Module Pass Counts

Table B.1 lists, for every module task, the number of passing samples over generated samples in five representative runs (labels per Table 4.2). This is the tabular form of the heatmap in Figure 5.10; the table body is generated directly from each run's records by `scripts/thesis_figures/gen_module_table.py`.

Table B.1: Passing samples / generated samples per module task. "–" = task absent from the run. Columns straddle the June-25 S-box testbench repair (Section 5.9); ROUTER-V1 postdates it and the other four columns predate it, so the `aes_sbox`/`aes_inv_sbox` rows are not comparable across columns.

Module	DIRECT-T0	PIPE-T0	CACHE-OFF	PIPE-120B	ROUTER-V1
`aes_cipher_top`	0/1	0/1	0/20	0/10	0/10
`aes_inv_cipher_top`	0/1	0/1	0/20	0/10	0/10
`aes_inv_sbox`	0/1	0/1	0/20	0/10	6/10
`aes_key_expand_128`	0/1	1/1	20/20	10/10	4/10
`aes_rcon`	1/1	1/1	6/20	5/10	2/10
`aes_sbox`	0/1	0/1	0/20	0/10	3/10
`sd_bd`	0/1	0/1	0/20	0/10	0/10
`sd_clock_divider`	1/1	1/1	0/20	0/10	0/10

doi:10.6342/NTU202601985

Module	Direct-t0	Pipe-t0	Cache-off	Pipe-120b	Router-v1
sd_cmd_master	0/1	0/1	0/20	0/10	0/10
sd_cmd_serial_host	0/1	0/1	0/20	0/10	0/10
sd_controller_wb	0/1	0/1	0/20	0/10	0/10
sd_crc_16	1/1	1/1	18/20	8/10	10/10
sd_crc_7	1/1	1/1	18/20	9/10	10/10
sd_data_master	0/1	0/1	0/20	0/10	0/10
sd_data_serial_host	0/1	0/1	0/20	0/10	0/10
sd_fifo_rx_filler	0/1	0/1	1/20	0/10	0/10
sd_fifo_tx_filler	0/1	1/1	19/20	8/10	10/10
sd_rx_fifo	0/1	0/1	0/20	0/10	0/10
sd_tx_fifo	0/1	0/1	0/20	0/10	0/10
sdc_controller	0/1	0/1	0/20	0/10	0/10
e203_biu	0/1	0/1	0/20	0/10	0/10
e203_clk_ctrl	0/1	1/1	9/20	1/10	7/10
e203_clkgate	1/1	1/1	20/20	10/10	10/10
e203_core	0/1	0/1	0/20	0/10	0/10
e203_cpu	0/1	0/1	0/20	0/10	0/10
e203_cpu_top	0/1	0/1	0/20	0/10	0/10
e203_dtcm_ctrl	0/1	0/1	1/20	5/10	4/10
e203_dtcm_ram	0/1	1/1	10/20	10/10	8/10
e203_extend_csr	0/1	1/1	15/20	9/10	6/10
e203_exu	0/1	0/1	0/20	0/10	0/10
e203_exu_alu	0/1	0/1	0/20	0/10	0/10
e203_exu_alu_bjp	0/1	0/1	2/20	2/10	0/10
e203_exu_alu_csrctrl	0/1	0/1	0/20	4/10	2/10
e203_exu_alu_dpath	0/1	0/1	0/20	0/10	0/10
e203_exu_alu_lsuagu	0/1	0/1	0/20	0/10	0/10
e203_exu_alu_muldiv	0/1	0/1	0/20	0/10	0/10
e203_exu_alu_rglr	0/1	0/1	5/20	7/10	5/10
e203_exu_branchslv	0/1	0/1	0/20	0/10	0/10

doi:10.6342/NTU202601985

Module	DIRECT-T0	PIPE-T0	CACHE-OFF	PIPE-120B	ROUTER-V1
e203_exu_commit	0/1	0/1	0/20	1/10	0/10
e203_exu_csr	0/1	0/1	0/20	0/10	0/10
e203_exu_decode	0/1	0/1	0/20	0/10	0/10
e203_exu_disp	0/1	0/1	0/20	0/10	1/10
e203_exu_excp	0/1	0/1	0/20	0/10	0/10
e203_exu_longpwbck	0/1	0/1	0/20	0/10	0/10
e203_exu_nice	0/1	0/1	0/20	0/10	0/10
e203_exu_oitf	0/1	0/1	0/20	3/10	0/10
e203_exu_regfile	0/1	1/1	2/20	5/10	6/10
e203_exu_wbck	0/1	0/1	0/20	0/10	0/10
e203_ifu	0/1	0/1	14/20	10/10	5/10
e203_ifu_ifetch	0/1	0/1	0/20	0/10	0/10
e203_ifu_ift2icb	0/1	0/1	0/20	0/10	0/10
e203_ifu_litebpu	0/1	0/1	0/20	0/10	0/10
e203_ifu_minidec	0/1	1/1	20/20	10/10	9/10
e203_irq_sync	0/1	1/1	14/20	10/10	7/10
e203_itcm_ctrl	0/1	0/1	0/20	0/10	0/10
e203_itcm_ram	0/1	1/1	8/20	10/10	5/10
e203_lsu	0/1	0/1	0/20	0/10	0/10
e203_lsu_ctrl	0/1	0/1	0/20	0/10	0/10
e203_reset_ctrl	0/1	0/1	0/20	0/10	0/10
e203_srams	0/1	1/1	20/20	10/10	10/10

B.2 Reproduction Commands

All runners live under `scripts/` in the project repository and require a live vLLM endpoint (`VLLM_BASE_URL`, `VLLM_MODEL`) plus Verilator on `PATH`. The commands below

doi:10.6342/NTU202601985

reproduce the thesis's principal arms; output directories match Table 4.3.

```
# Headline pipeline (temp 0) and pure-model baseline
python scripts/run_agentic_plan_legacy_realbench.py \
  --task-level module --samples 1 --temperature 0 \
  --output-dir runs/agentic_plan_legacy_realbench_plan_hallu_fix_t0
python scripts/run_realbench_direct_model.py \
  --task-level module --samples 1 --temperature 0 \
  --output-dir runs/realbench_direct_model

# Repair-cache ablation (swap off | task | run)
python scripts/run_agentic_plan_legacy_realbench.py \
  --task-level module --samples 20 \
  --planner-search-mode hybrid --repair-cache task \
  --output-dir runs/..._task_rep_cache

# Grounding single-fix ablations
# --no-planner-inject-catalog | --no-planner-ground-reuse |
# --no-planner-completeness-gate

# Full cascade router, original v1 rule (Tier-0 LLM decider + plan probe)
python scripts/run_realbench_routed.py \
  --router cascade --decider llm --route-rule v1 --samples 10 \
  --output-dir runs/Full-2T_router_oss20B_sync6_func4

# Re-derived rule (Router-v2: planning-default carve-out, no probe)
python scripts/run_realbench_routed.py \
  --router cascade --decider llm --route-rule v2-20b --samples 10 \
  --output-dir runs/Full-2T_router_oss20B_syn6_func4_s10_t02_v2

# Per-module pass tables and figures
python scripts/module_pass_rate_report.py runs/<run_dir> \
  -o <run_dir>/module_pass_rate_analysis.md
```


doi:10.6342/NTU202601985

```
cd scripts/thesis_figures && for f in fig_*.py; do python $f; done
```

Listing B.12: Reproducing the principal experimental arms.

doi:10.6342/NTU202601985